三维点云处理技术
和深度学习在点云处理中的应用

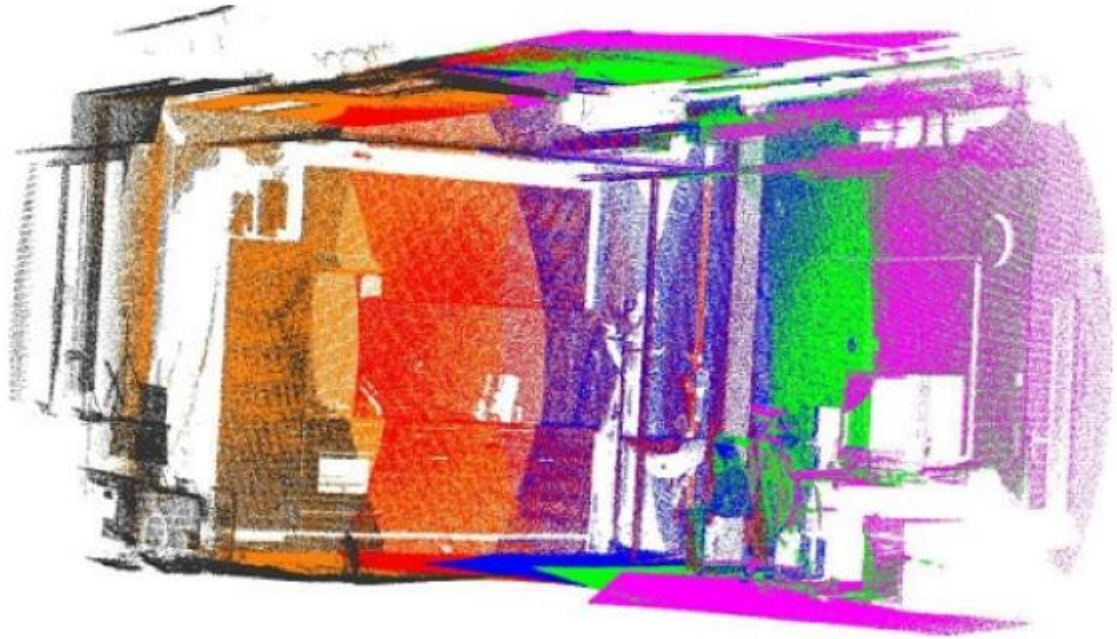
三维点云配准与点云SLAM基础

索传哲

内容概要

点云配准方法
SLAM基础框架
帧间匹配与激光里程计
SLAM图优化基础

点云配准方法



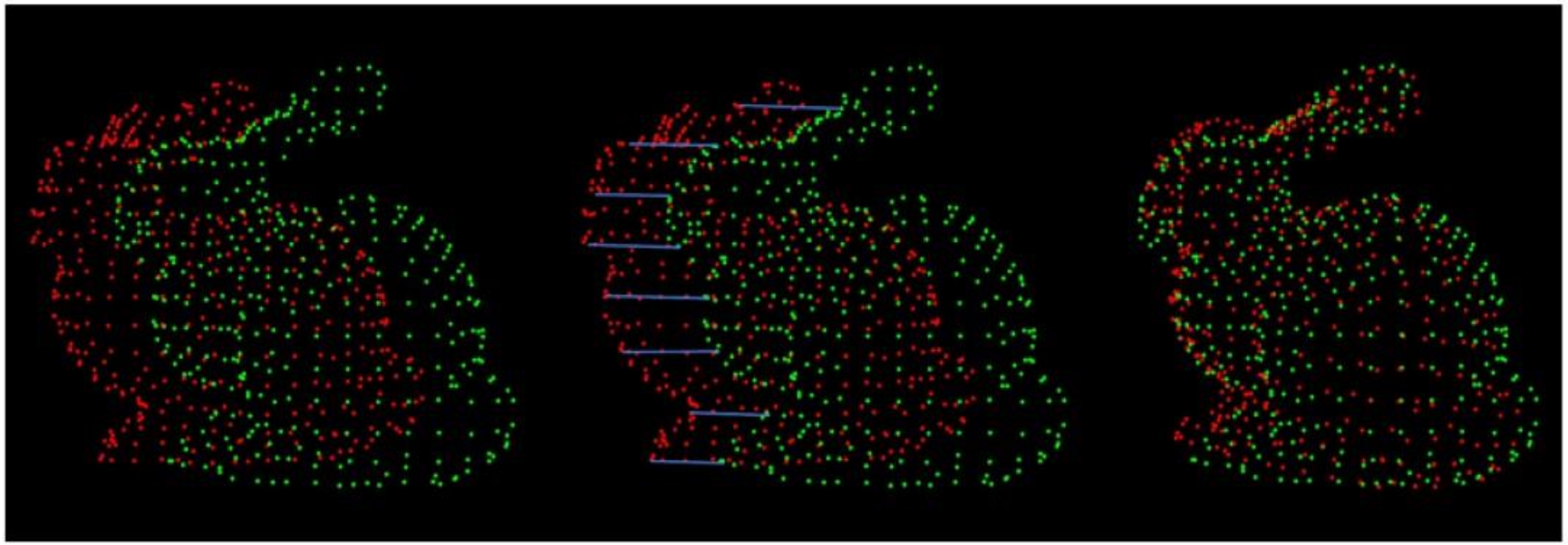
ICP迭代修正两个原始点云的刚体变换(平移、旋转)，以最小化所有点集之间的距离。

- 输入：两帧原始点云、变换的初始估计、停止迭代的标准
- 输出：变换矩阵、变换后的修正点云

Jochen Sprickerhof:PCL::Registration

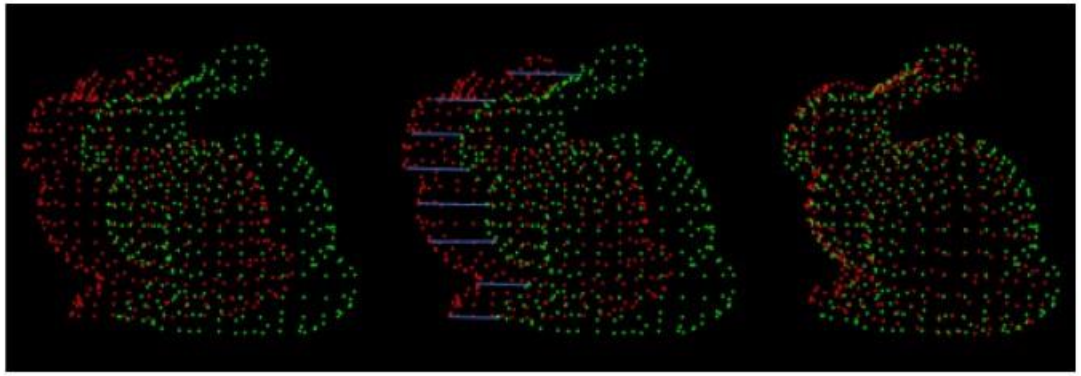
点云配准方法

- 点云配准
  - 给定原始点云与目标点云
    - 确定匹配对关系
    - 估计变换矩阵以对齐匹配对
    - 应用该变换以对齐原始点云与目标点云
    - 迭代!



Jochen Sprickerhof:PCL::Registration

点云配准方法



- 点云配准

PCL ICP:

Code

```
pcl::IterativeClosestPoint<pcl::PointXYZ, pcl::PointXYZ> icp;
icp.setInputCloud(cloud_in);
icp.setInputTarget(cloud_out);
pcl::PointCloud<pcl::PointXYZ> Final;
icp.align(Final);
```

$$\Delta T = \arg\min_{R,t} \sum_{i=1}^{n} \left\| p_{bi} - (R p'_{ai} + t) \right\|_2^2$$

$$\Delta T = \begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix}$$

ICP 算法步骤:

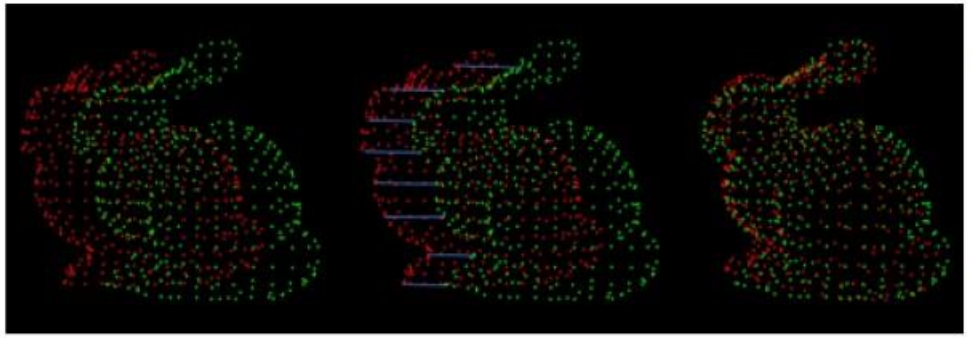
- (1)对点集A中每一个点*Pai*施加初始变换*T0*，得到*Pai'*；
- (2)从点集B中寻找距离点*Pai'*最近的点*Pbi*，形成对应的点对；
- (3)求解最优变换ΔT
- (4)根据前后两次的迭代误差以及迭代次数等条件判断是否收敛。如果收敛，则输出最终结果:$T=\Delta T*T0$，否则$T0=\Delta T*T0$,重复步骤(1)；

ICP API：终止条件

- 最大迭代次数
  - 通过 `setMaximumIterations(nr_iterations)` 设置
- 收敛标准：估计点云变换不再变化（当前变换和上次变换的差值之和小于用户定义的阈值）
  - 通过 `setTransformationEpsilon(epsilon)` 设置
- 找到的解（误差平方和小于用户自定义的阈值）
  - 通过 `setEuclideanFitnessEpsilon(distance)` 设置

Jochen Sprickerhof:PCL::Registration；泡泡机器人

点云配准方法



- 点云配准

PCL ICP:

常用ICP接口

```
pcl::IterativeClosestPoint<PointT, PointT> icp;
//设定最大迭代次数，超过此次数ICP终止
icp.setMaximumIterations (iterations);
//输入源点云，即被转换的点云
icp.setInputSource (cloud_icp);
//输入目标点云，即参考点云
icp.setInputTarget (cloud_in);
//设置查找近邻时是否双向搜索
icp.setUseReciprocalCorrespondences(false);
//设置最大近邻点距离，超过此距离的点对将不参与计算
icp.setMaxCorrespondenceDistance(0.5);
//设置匹配对剔除器
 icp.addCorrespondenceRejector(rejector);
```

```
//设置迭代误差收敛阈值
icp.setEuclideanFitnessEpsilon(0.0001);
//设置ICP计算方法，可以为SVD、LM…
icp.setTransformationEstimation(te);
//执行运算，ICP的主要运算都在align函数中
icp.align (*cloud_icp);
//获取ICP是否收敛
icp.hasConverged ();
//获取ICP匹配得到的变换矩阵T，
Eigen::Matrix4f transformation_matrix =
icp.getFinalTransformation ();

//获取ICP匹配得分
icp.getFitnessScore();
```

Jochen Sprickerhof:PCL::Registration；泡泡机器人

点云配准方法

- 点云配准

PCL ICP:

ICP实现分析（align）

align()函数是ICP算法的父类Registration类定义的一个函数，来实现配准算法的统一接口调用。align()函数中调用computeTransformation()虚方法，ICP重载该方法来具体实现变换矩阵的解算方法。

(1) transformCloud(*input_,*input_transformed,guess);

根据初始变换估计矩阵guess进行坐标转换

void align(PointCloudSource &output, const Matrix4& guess);

(2)correspondence_estimation_->determineCorrespondences(*correspondences_, corr_dist_threshold_);

利用kdtree寻找近邻点建立对应关系。

*correspondence_estimation_->determineReciprocalCorrespondences (*correspondences_, corr_dist_threshold_);*

即双向搜索方法

*setUseReciprocalCorrespondences(bool use_reciprocal_correspondence)*函数确定是否启用

(3)rej->getCorrespondences (*correspondences_);

对应关系剔除器

(4)transformation_estimation_->estimateRigidTransformation(*input_transformed, *target_, *correspondences_, transformation_);

根据对应关系计算变换矩阵T：SVD分解(默认)，LM优化等

transformation_estimation_.reset(new pcl::registration::TransformationEstimationSVD<PointSource, PointTarget, Scalar> ());

void setTransformationEstimation(const TransformationEstimationPtr &te)

例如使用LM方法求解ICP：

pcl::registration::TransformationEstimationLM<Pt,Pt,float>::Ptr te_lm_ (new pcl::registration:: TransformationEstimationLM<Pt,Pt,float>) ;

icp.setTransformationEstimation(te_lm_);

(5)final_transformation_ = transformation_ * final_transformation_;

迭代叠加变换矩阵transformation

(6)converged_ = static_cast<bool> ((*convergence_criteria_));

判断是否收敛。

是否超过最大迭代次数

判断前后两次迭代的旋转以及平移差是否小于某个阈值

判断前后两次迭代“平均距离”之差是否小于某个阈值。

点云配准方法

- 点云配准

PCL ICP:

SVD估计变换原理:

$$\mathcal{P} = \{\mathbf{p}_1, \mathbf{p}_2, \ldots, \mathbf{p}_n\}$$
$$\mathcal{Q} = \{\mathbf{q}_1, \mathbf{q}_2, \ldots, \mathbf{q}_n\}$$

$$(R, \mathbf{t}) = \underset{R \in SO(d),\ \mathbf{t} \in \mathbb{R}^d}{\operatorname{argmin}} \sum_{i=1}^{n} w_i \left\| (R\mathbf{p}_i + \mathbf{t}) - \mathbf{q}_i \right\|^2$$

$$F(\mathbf{t}) = \sum_{i=1}^{n} w_i \left\| (R\mathbf{p}_i + \mathbf{t}) - \mathbf{q}_i \right\|^2$$

$$0 = \frac{\partial F}{\partial \mathbf{t}} = \sum_{i=1}^{n} 2w_i \left(R\mathbf{p}_i + \mathbf{t} - \mathbf{q}_i\right) =$$

$$= 2\mathbf{t} \left(\sum_{i=1}^{n} w_i \right) + 2R \left(\sum_{i=1}^{n} w_i \mathbf{p}_i \right) - 2 \sum_{i=1}^{n} w_i \mathbf{q}_i$$

$$\bar{\mathbf{p}} = \frac{\sum_{i=1}^{n} w_i \mathbf{p}_i}{\sum_{i=1}^{n} w_i}, \quad \bar{\mathbf{q}} = \frac{\sum_{i=1}^{n} w_i \mathbf{q}_i}{\sum_{i=1}^{n} w_i}$$

$$\mathbf{t} = \bar{\mathbf{q}} - R\bar{\mathbf{p}}.$$

$$\sum_{i=1}^{n} w_i \left\| (R\mathbf{p}_i + \mathbf{t}) - \mathbf{q}_i \right\|^2 = \sum_{i=1}^{n} w_i \left\| R\mathbf{p}_i + \bar{\mathbf{q}} - R\bar{\mathbf{p}} - \mathbf{q}_i \right\|^2 =$$

$$= \sum_{i=1}^{n} w_i \left\| R(\mathbf{p}_i - \bar{\mathbf{p}}) - (\mathbf{q}_i - \bar{\mathbf{q}}) \right\|^2.$$

$$\mathbf{x}_i := \mathbf{p}_i - \bar{\mathbf{p}}, \quad \mathbf{y}_i := \mathbf{q}_i - \bar{\mathbf{q}}.$$

$$R = \underset{R \in SO(d)}{\operatorname{argmin}} \sum_{i=1}^{n} w_i \left\| R\mathbf{x}_i - \mathbf{y}_i \right\|^2$$

Sorkine O, Rabinovich M. Least-squares rigid motion using svd, ETHZ

点云配准方法

- 点云配准

PCL ICP:

SVD估计变换原理:

$$\begin{aligned}\|R\mathbf{x}_i-\mathbf{y}_i\|^2 &= (R\mathbf{x}_i-\mathbf{y}_i)^{\mathsf{T}}(R\mathbf{x}_i-\mathbf{y}_i)=(\mathbf{x}_i^{\mathsf{T}}R^{\mathsf{T}}-\mathbf{y}_i^{\mathsf{T}})(R\mathbf{x}_i-\mathbf{y}_i)= \\ &= \mathbf{x}_i^{\mathsf{T}}R^{\mathsf{T}}R\mathbf{x}_i-\mathbf{y}_i^{\mathsf{T}}R\mathbf{x}_i-\mathbf{x}_i^{\mathsf{T}}R^{\mathsf{T}}\mathbf{y}_i+\mathbf{y}_i^{\mathsf{T}}\mathbf{y}_i= \\ &= \mathbf{x}_i^{\mathsf{T}}\mathbf{x}_i-\mathbf{y}_i^{\mathsf{T}}R\mathbf{x}_i-\mathbf{x}_i^{\mathsf{T}}R^{\mathsf{T}}\mathbf{y}_i+\mathbf{y}_i^{\mathsf{T}}\mathbf{y}_i.\end{aligned}$$

$$\mathbf{x}_i^{\mathsf{T}}R^{\mathsf{T}}\mathbf{y}_i=(\mathbf{x}_i^{\mathsf{T}}R^{\mathsf{T}}\mathbf{y}_i)^{\mathsf{T}}=\mathbf{y}_i^{\mathsf{T}}R\mathbf{x}_i \qquad \|R\mathbf{x}_i-\mathbf{y}_i\|^2=\mathbf{x}_i^{\mathsf{T}}\mathbf{x}_i-2\mathbf{y}_i^{\mathsf{T}}R\mathbf{x}_i+\mathbf{y}_i^{\mathsf{T}}\mathbf{y}_i.$$

$$\underset{R\in SO(d)}{\operatorname{argmin}}\sum_{i=1}^{n}w_i\|R\mathbf{x}_i-\mathbf{y}_i\|^2=\underset{R\in SO(d)}{\operatorname{argmin}}\sum_{i=1}^{n}w_i(\mathbf{x}_i^{\mathsf{T}}\mathbf{x}_i-2\mathbf{y}_i^{\mathsf{T}}R\mathbf{x}_i+\mathbf{y}_i^{\mathsf{T}}\mathbf{y}_i)=$$

$$=\underset{R\in SO(d)}{\operatorname{argmin}}\left(\sum_{i=1}^{n}w_i\mathbf{x}_i^{\mathsf{T}}\mathbf{x}_i-2\sum_{i=1}^{n}w_i\mathbf{y}_i^{\mathsf{T}}R\mathbf{x}_i+\sum_{i=1}^{n}w_i\mathbf{y}_i^{\mathsf{T}}\mathbf{y}_i\right)=$$

$$=\underset{R\in SO(d)}{\operatorname{argmin}}\left(-2\sum_{i=1}^{n}w_i\mathbf{y}_i^{\mathsf{T}}R\mathbf{x}_i\right).$$

$$\underset{R\in SO(d)}{\operatorname{argmin}}\left(-2\sum_{i=1}^{n}w_i\mathbf{y}_i^{\mathsf{T}}R\mathbf{x}_i\right)=\underset{R\in SO(d)}{\operatorname{argmax}}\sum_{i=1}^{n}w_i\mathbf{y}_i^{\mathsf{T}}R\mathbf{x}_i$$

Sorkine O, Rabinovich M. Least-squares rigid motion using svd, ETHZ

点云配准方法

- 点云配准

PCL ICP:

SVD估计变换原理:

$$R = \underset{R \in SO(d)}{\operatorname{argmin}} \sum_{i=1}^{n} w_i \left\| R\mathbf{x}_i - \mathbf{y}_i \right\|^2 \quad \underset{R \in SO(d)}{\operatorname{argmin}} \left(-2 \sum_{i=1}^{n} w_i \mathbf{y}_i^{\mathsf{T}} R \mathbf{x}_i \right) = \underset{R \in SO(d)}{\operatorname{argmax}} \sum_{i=1}^{n} w_i \mathbf{y}_i^{\mathsf{T}} R \mathbf{x}_i$$

$$\begin{bmatrix} w_1 & & & \\ & w_2 & & \\ & & \ddots & \\ & & & w_n \end{bmatrix} \begin{bmatrix} — & \mathbf{y}_1^{\mathsf{T}} & — \\ — & \mathbf{y}_2^{\mathsf{T}} & — \\ & \vdots & \\ — & \mathbf{y}_n^{\mathsf{T}} & — \end{bmatrix} \begin{bmatrix} & & \\ & R & \\ & & \end{bmatrix} \begin{bmatrix} | & | & & | \\ \mathbf{x}_1 & \mathbf{x}_2 & \dots & \mathbf{x}_n \\ | & | & & | \end{bmatrix} =$$

$$= \begin{bmatrix} — & w_1 \mathbf{y}_1^{\mathsf{T}} & — \\ — & w_2 \mathbf{y}_2^{\mathsf{T}} & — \\ & \vdots & \\ — & w_n \mathbf{y}_n^{\mathsf{T}} & — \end{bmatrix} \begin{bmatrix} | & | & & | \\ R\mathbf{x}_1 & R\mathbf{x}_2 & \dots & R\mathbf{x}_n \\ | & | & & | \end{bmatrix} = \begin{bmatrix} w_1 \mathbf{y}_1^{\mathsf{T}} R \mathbf{x}_1 & & & * \\ & w_2 \mathbf{y}_2^{\mathsf{T}} R \mathbf{x}_2 & & \\ & & \ddots & \\ * & & & w_n \mathbf{y}_n^{\mathsf{T}} R \mathbf{x}_n \end{bmatrix}$$

$$\sum_{i=1}^{n} w_i \mathbf{y}_i^{\mathsf{T}} R \mathbf{x}_i = \operatorname{tr}\left(W Y^{\mathsf{T}} R X \right)$$

Sorkine O, Rabinovich M. Least-squares rigid motion using svd, ETHZ

点云配准方法

- 点云配准

PCL ICP:

SVD估计变换原理:

$$\sum_{i=1}^{n} w_i \mathbf{y}_i^{\mathsf{T}} R \mathbf{x}_i = \operatorname{tr}\left(WY^{\mathsf{T}}RX\right) \qquad \operatorname{tr}\left(WY^{\mathsf{T}}RX\right) = \operatorname{tr}\left((WY^{\mathsf{T}})(RX)\right) = \operatorname{tr}\left(RXWY^{\mathsf{T}}\right)$$

$$S = XWY^{\mathsf{T}} \qquad S = U\Sigma V^{\mathsf{T}} \qquad \operatorname{tr}\left(RXWY^{\mathsf{T}}\right) = \operatorname{tr}(RS) = \operatorname{tr}\left(RU\Sigma V^{\mathsf{T}}\right) = \operatorname{tr}\left(\Sigma V^{\mathsf{T}}RU\right)$$

$$M = V^{\mathsf{T}}RU \qquad 1 = \mathbf{m}_j^{\mathsf{T}}\mathbf{m}_j = \sum_{i=1}^{d} m_{ij}^2 \;\Rightarrow\; m_{ij}^2 \le 1 \;\Rightarrow\; |m_{ij}| \le 1$$

$$\operatorname{tr}(\Sigma M) = \begin{pmatrix} \sigma_1 & & & \\ & \sigma_2 & & \\ & & \ddots & \\ & & & \sigma_d \end{pmatrix} \begin{pmatrix} m_{11} & m_{12} & \dots & m_{1d} \\ m_{21} & m_{22} & \dots & m_{2d} \\ \vdots & \vdots & \vdots & \vdots \\ m_{d1} & m_{d2} & \dots & m_{dd} \end{pmatrix} = \sum_{i=1}^{d} \sigma_i m_{ii} \le \sum_{i=1}^{d} \sigma_i$$

$$I = M = V^{\mathsf{T}}RU \;\Rightarrow\; V = RU \;\Rightarrow\; R = VU^{\mathsf{T}}$$

$$\det(VU^{\mathsf{T}}) = +1$$

$$\det(VU^{\mathsf{T}}) = -1$$

$$R = V \begin{pmatrix} 1 & & & \\ & 1 & & \\ & & \ddots & \\ & & & \det(VU^{\mathsf{T}}) \end{pmatrix} U^{\mathsf{T}}$$

Sorkine O, Rabinovich M. Least-squares rigid motion using svd, ETHZ

点云配准方法

- 点云配准

PCL ICP:

SVD求解变换流程:

- 求R|t使得原始点云与目标点云距离最小

$$\sum_{i=1}^{n} w_i \left\| (R\mathbf{p}_i + \mathbf{t}) - \mathbf{q}_i \right\|^2$$

- 计算两个点云集的加权质心

$$\bar{\mathbf{p}} = \frac{\sum_{i=1}^{n} w_i \mathbf{p}_i}{\sum_{i=1}^{n} w_i}, \quad \bar{\mathbf{q}} = \frac{\sum_{i=1}^{n} w_i \mathbf{q}_i}{\sum_{i=1}^{n} w_i}$$

- 计算中心化向量

$$\mathbf{x}_i := \mathbf{p}_i - \bar{\mathbf{p}}, \quad \mathbf{y}_i := \mathbf{q}_i - \bar{\mathbf{q}}, \qquad i = 1, 2, \ldots, n$$

- 计算d*d的协方差矩阵, X,Y:d*n

$$S = XWY^{\mathsf{T}}$$

- 奇异值分解，求解R

$$S = U\Sigma V^{\mathsf{T}}$$

$$R = V \begin{pmatrix} 1 & & & \\ & 1 & & \\ & & \ddots & \\ & & & \det(VU^{\mathsf{T}}) \end{pmatrix} U^{\mathsf{T}}$$

- 计算最佳的平移t

$$\mathbf{t} = \bar{\mathbf{q}} - R\bar{\mathbf{p}}$$

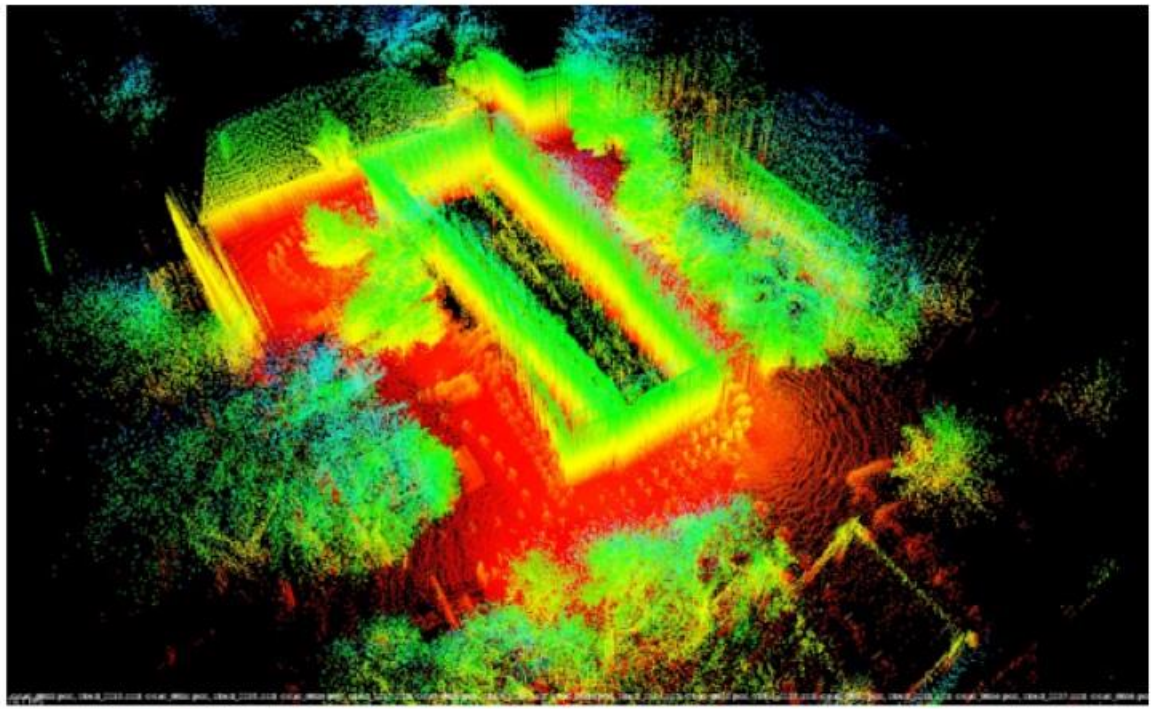
Sorkine O, Rabinovich M. Least-squares rigid motion using svd, ETHZ

点云配准方法

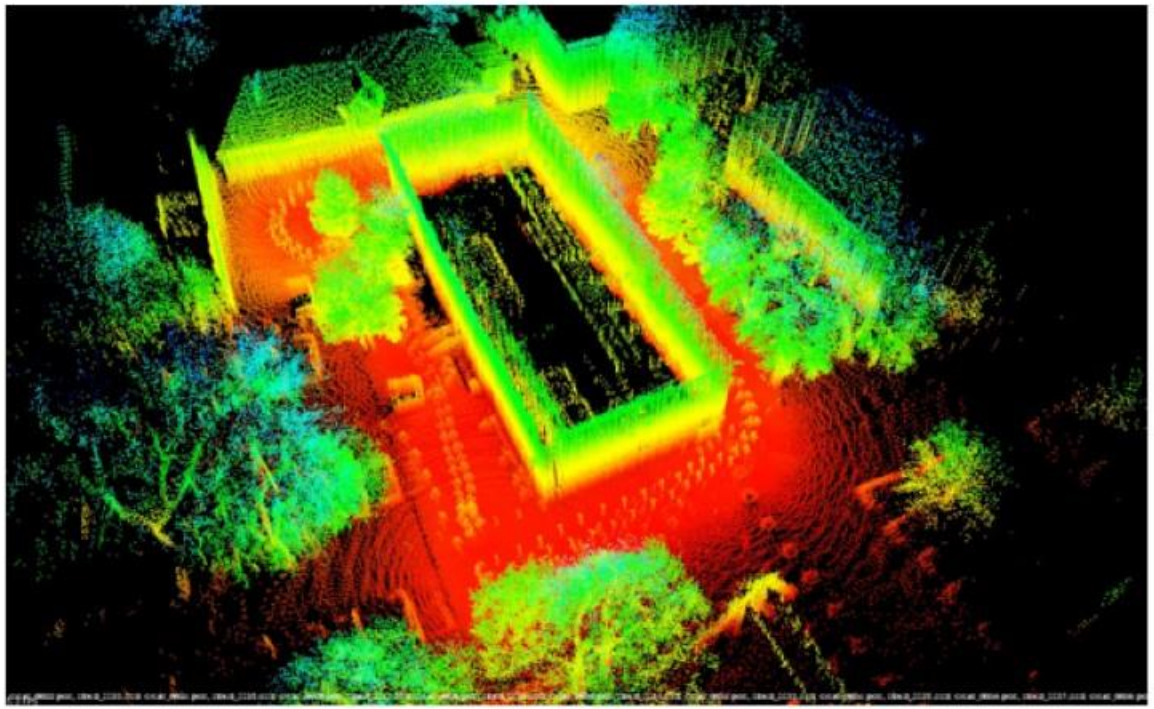
- 点云配准

PCL ICP:

Lidar SLAM前端：



Input Data



pcl_icp -d 0.75 -r 0.75 -i 50 ../data/cloud_0{022..130}.pcd

Jochen Sprickerhof:PCL::Registration

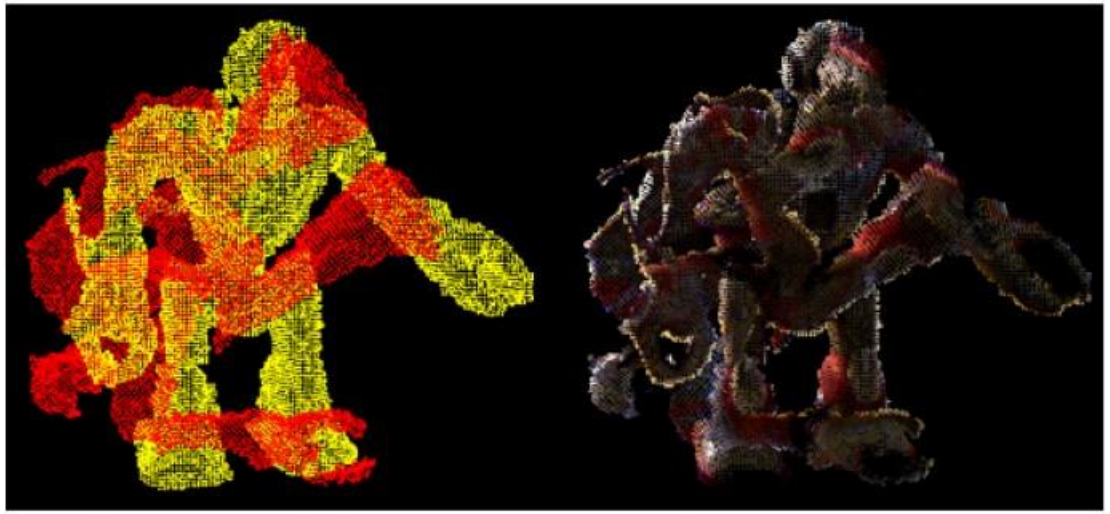
点云配准方法

- 点云配准

PCL ICP:

Transformation Estimation:

```
Eigen::Matrix4f transformation;
TransformationEstimationSVD<PointT, PointT> svd;
svd.estimateRigidTransformation(src, trgt, corres, trans);
```



- 示例：简单初始对齐（Initial alignment）
- 问题：虚假对齐

icp缺点：

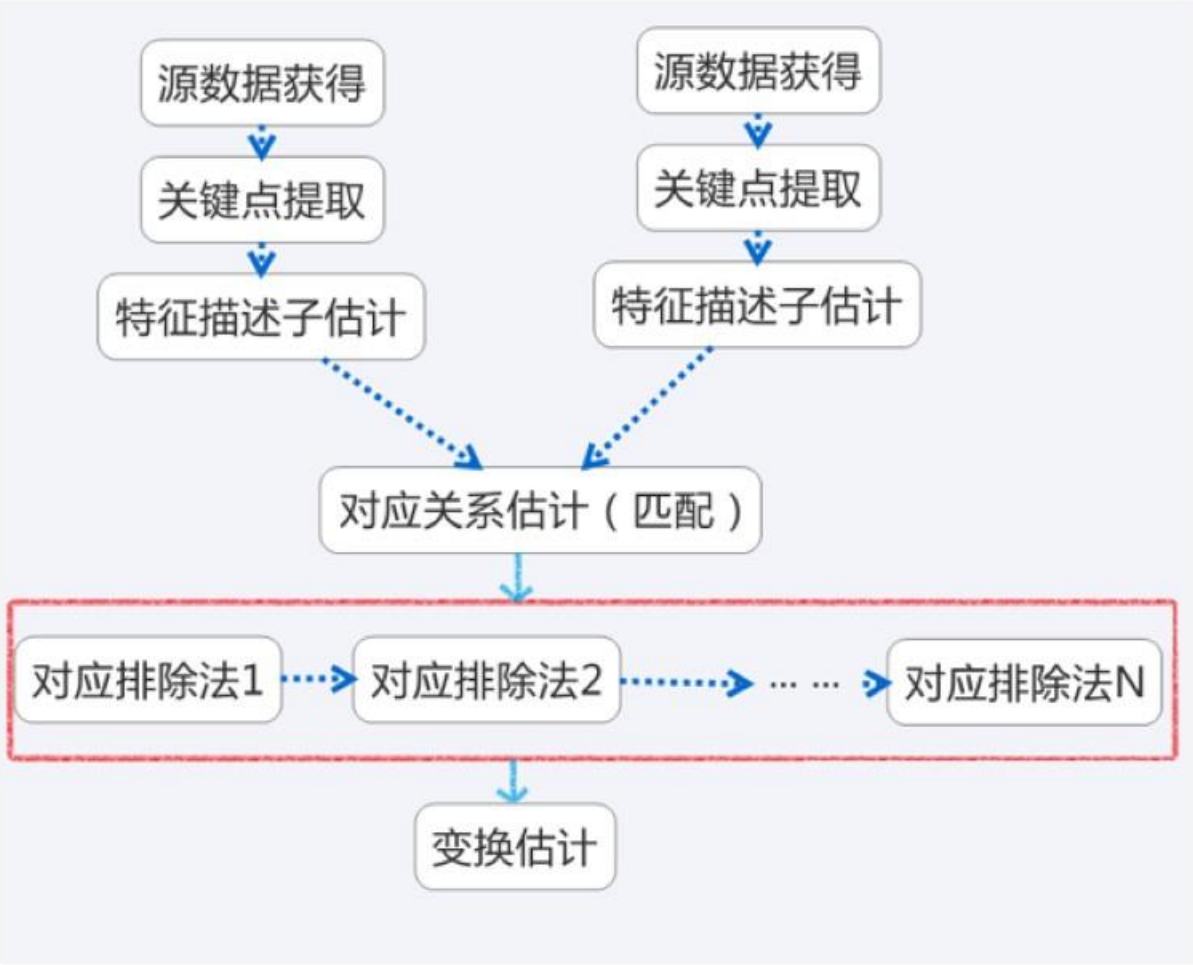
- 算法收敛于局部最小误差。
- 噪声或异常数据可能导致算法无法收敛或错误。
- 在进行ICP算法第一步要确定一个迭代初值，选取的初值将对最后配准结果产生重要的影响，如果初值选择不合适，算法可能就会限入局部最优

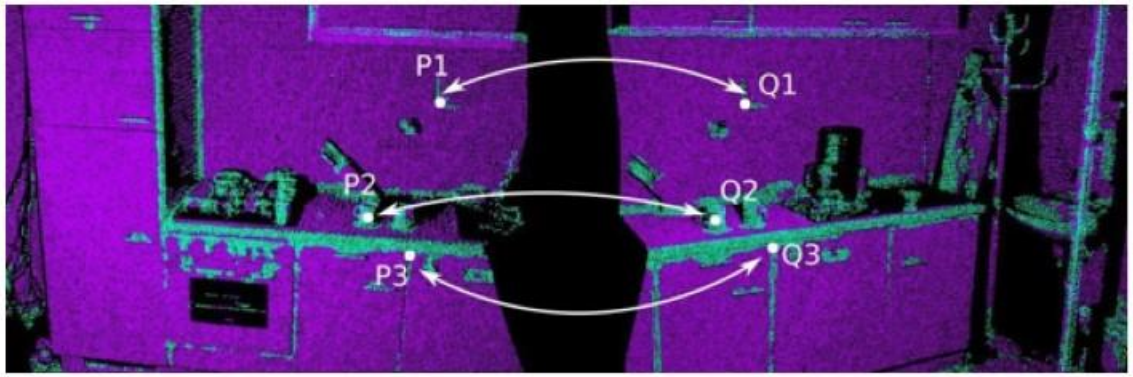
Jochen Sprickerhof:PCL::Registration

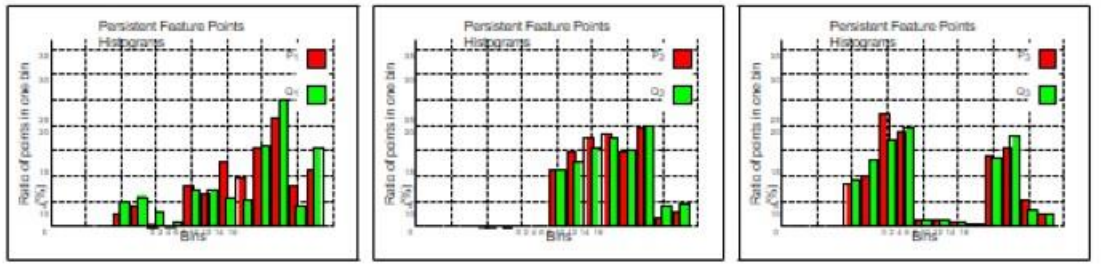
点云配准方法

- 点云配准

广义配准Registration：







对两个点云a，b匹配运算步骤如下：：

- 采集原始点云，分析提取两个原点云关键点k
- 在每个关键点ki处，计算相应的特征描述子fi
- 根据特征描述子{fi}和相对应的XYZ坐标位置，估计对应匹配关系
- 因为噪声或虚假匹配，并非所有的对应关系都有效，根据规则排除虚假对应匹配
- 根据有效对应匹配关系，估计刚体变换关系

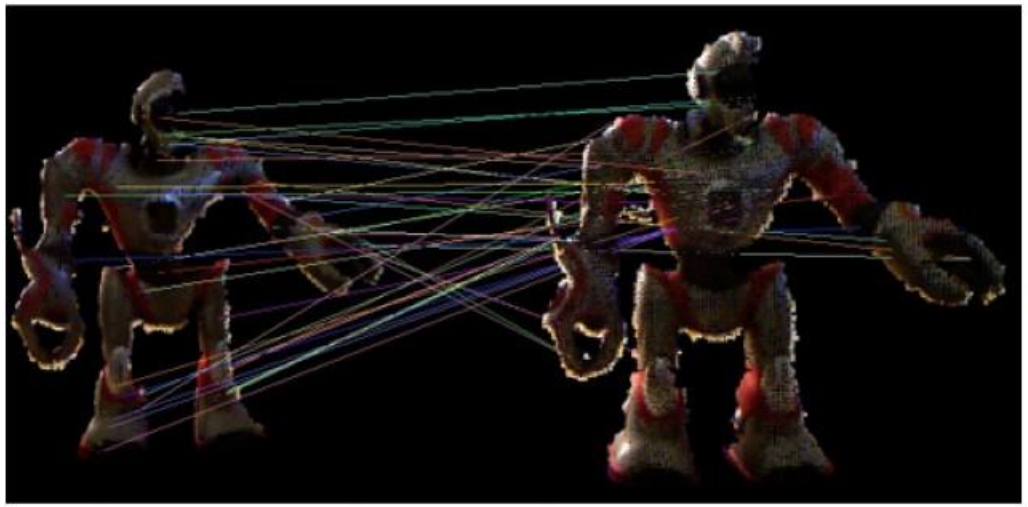
点云配准方法

- 点云配准

广义配准Registration：

Matching Features：

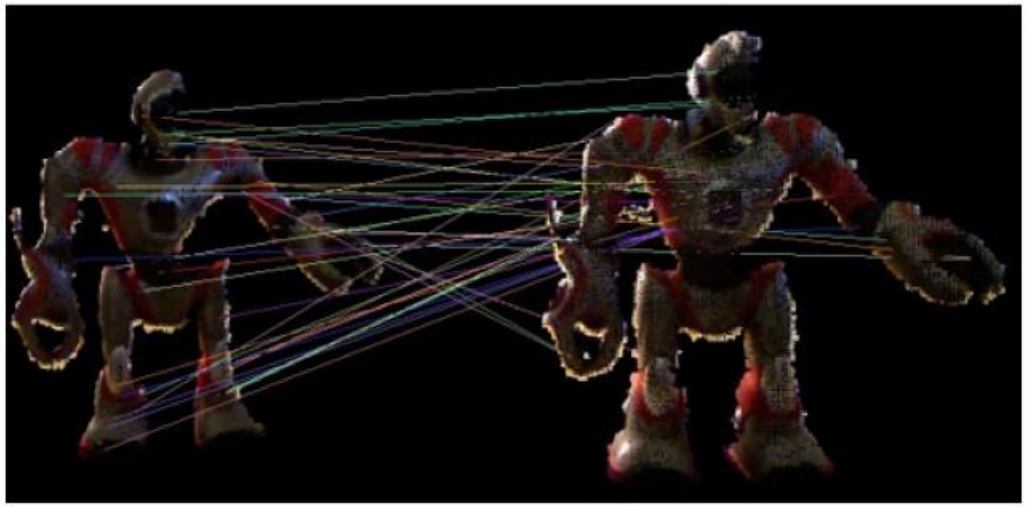
```
CorrespondenceEstimation<FeatureT, FeatureT> est;
est.setInputCloud (source_features);
est.setInputTarget (target_features);
est.determineCorrespondences (correspondences);
```



示例：寻找对应匹配（Find correspondences）

Jochen Sprickerhof:PCL::Registration

点云配准方法



- 点云配准

广义配准Registration：
用SAC排除虚假匹配（outliers）：

- 选择三对对应匹配对 d_i，m_i
- 估计选择样本的变换关系(R,t)
- 用$((Rd_i + t) - m_i)^2 < c$确定内点对
- 重复N次，使（R,t）有更多的内点
- 问题：在描述特征较少的情况下，最好的匹配m_i可能不是d_i真正的对应

用SAC排除虚假匹配：

```
CorrespondenceRejectorSampleConsensus<PointT> sac;
sac.setInputCloud(source);
sac.setTargetCloud(target);
sac.setInlierThreshold(epsilon);
sac.setMaxIterations(N);
sac.setInputCorrespondences(correspondences);
sac.getCorrespondences(inliers);
Eigen::Matrix4f transformation = sac.getBestTransformation();
```

Jochen Sprickerhof:PCL::Registration

点云配准方法

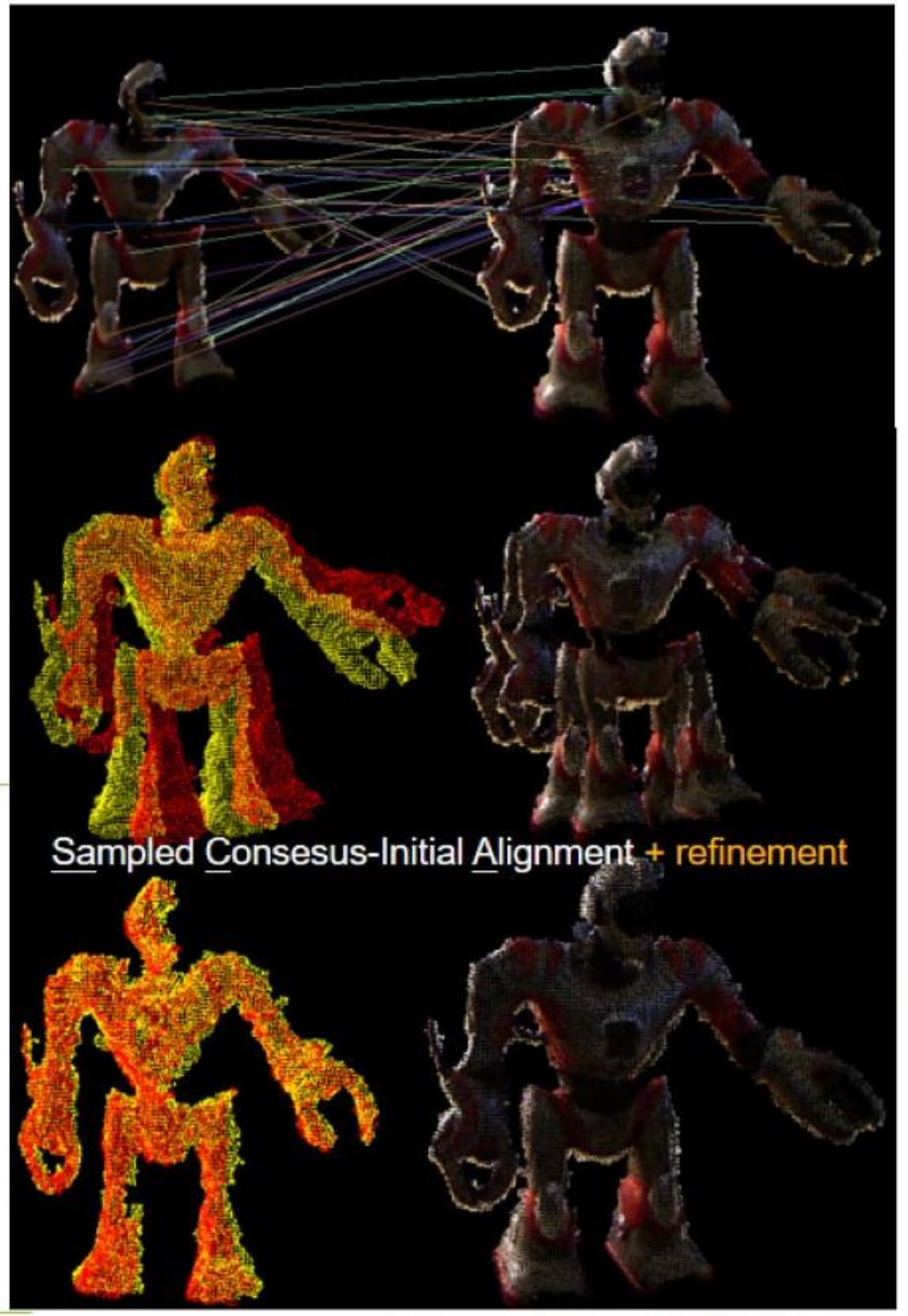
点云配准

广义配准Registration :

SAC-IA: Sampled Consesus-Initial Alignment

- 在原点云中选择n个点 d_i,
- 对每个点d_i:
  - 选择k个最近匹配
  - 并且选择其中一个为匹配点m_i
- 估计选择样本的变换关系(R,t)
- 用$((Rd_i + t) - m_i)^2 < c$确定内点对
- 重复N次，使（R,t）有更多的内点

```
pcl::SampleConsensusInitialAlignment
<PointT, PointT, FeatureT> sac_ia;
sac_ia.setNumberOfSamples(n);
sac_ia.setMinSampleDistance(d);
sac_ia.setCorrespondenceRandomness(k);
sac_ia.setMaximumIterations(N);
sac_ia.setInputCloud(source);
sac_ia.setInputTarget(target);
sac_ia.setSourceFeatures(source_features);
sac_ia.setTargetFeatures(target_features);
sac_ia.align(aligned_source);
Eigen::Matrix4f = sac_ia.getFinalTransformation();
```

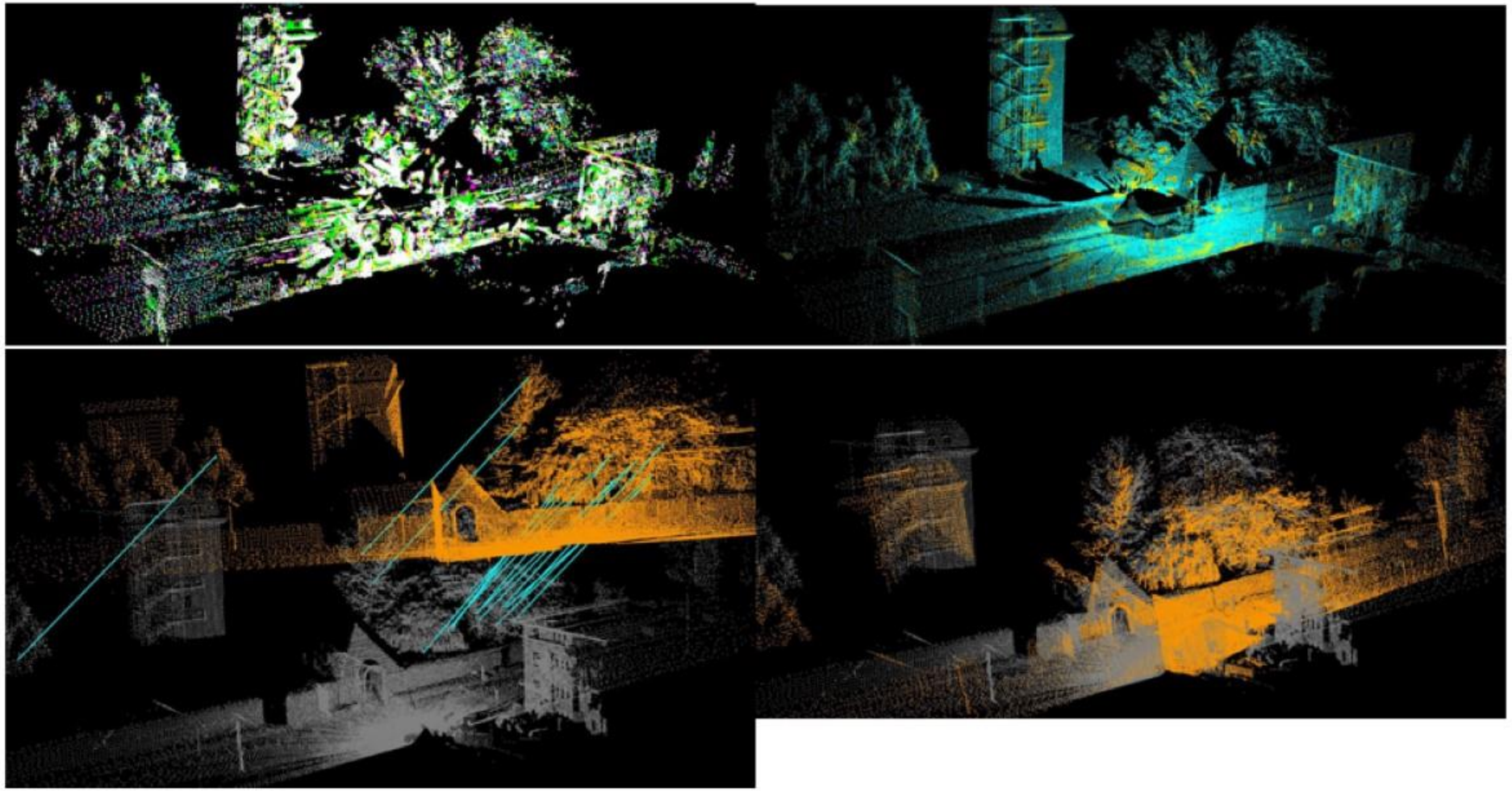


Jochen Sprickerhof:PCL::Registration

点云配准方法

- 点云配准

  广义配准Registration :



Jochen Sprickerhof:PCL::Registration

点云配准方法

- 点云配准

点云配准API:

基于对应3D匹配计算变换矩阵

CorrespondenceEstimation类
用来确定两点云中的对应匹配关系。

CorrespondenceRejectorFeatures类
能够实现基于特征匹配的对应点对提取。

SampleConsensusPrerejective类
实现了基于RANSAC方法的对应点对外点去除，用于姿态评估和点云配准中的几何一致性限制，从而保证后续算法的准确和高效。

TransformationEstimation3Point类
实现使用三个匹配点计算两点云变换矩阵。

TransformationFromCorrespondences类
实现三个及以上匹配点间变换矩阵的计算。

TransformationEstimationDQ类和
TransformationEstimationDualQuaternion类
实现基于对偶四元数的点云姿态估计。

TransformationEstimationLM类
实现基于对应匹配点对，用Levenberg-Marquardt优化的变换矩阵估计。

TransformationEstimationPointToPlane类
实现基于Levenberg-Marquardt优化的匹配点对间最小点到平面距离的最优变换矩阵计算。

TransformationEstimationPointToPlaneWeighted类
在TransformationEstimationPointToPlane类的基础上加入了对应匹配点的权重，以适应更为复杂场景点云配准任务。

TransformationEstimationPointToPlaneLLS 类
基于线性最小二乘的点到平面距离累计最小化约束，对应点到平面的距离计算加入法向量约束
参考：《Linear Least-Squares Optimization for Point-to-Plane ICP surface registration》 2014

TransformationEstimationPointToPlaneLLSWeighted类
在TransformationEstimationPointToPlaneLLS 类基础上加入了对应点匹配权重，从而更合理的计算匹配点间变换矩阵。

TransformationEstimationSVD类
使用SVD分解策略完成匹配点对的变换矩阵计算。

TransformationValidationEuclidean类
实现了给定对应匹配的变换，并计算可靠性得分。

点云配准方法

- 点云配准

点云配准API:

基于迭代最近点的配准方法

IterativeClosestPoint类

实现迭代最近点算法（ICP）。

IterativeClosestPointNonLinear类

ICP算法的变体，后端使用Levenberg-Marquardt作为优化。

GeneralizedIterativeClosestPoint类

ICP算法的变体，实现了一般化的ICP算法

参考：Generalized-ICP论文。

GeneralizedIterativeClosestPoint6D类

集成了L*a*b*颜色空间到Generalized-ICP算法中。

IterativeClosestPointWithNormals 类

ICP算法的变体，以点到平面的最小距离作为迭代优化对象，实现点云配准。

基于RANSAC框架的点云粗配准

FPCSInitialAlignment 类

实现《4-points congruent sets for robust pairwise surface registration》中的4PCS算法；

KFPCSInitialAlignment类

实现《Markerless point cloud registration with keypoint-based 4-points congruent sets》中的k4PCS算法

基于概率模型的点云配准方法

NormalDistributionsTransform（NDT）类

统计分析点云中法向量分布，来实现点云的转化和配准。

参考：《The Three-Dimensional Normal-Distributions Transform an Efficient Representation for Registration》2009

内容概要

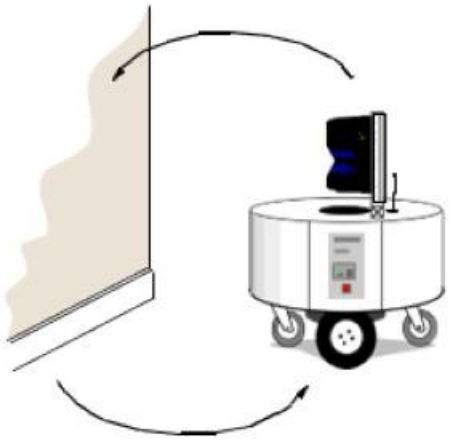
点云配准方法
SLAM基础框架
帧间匹配与激光里程计
SLAM图优化基础

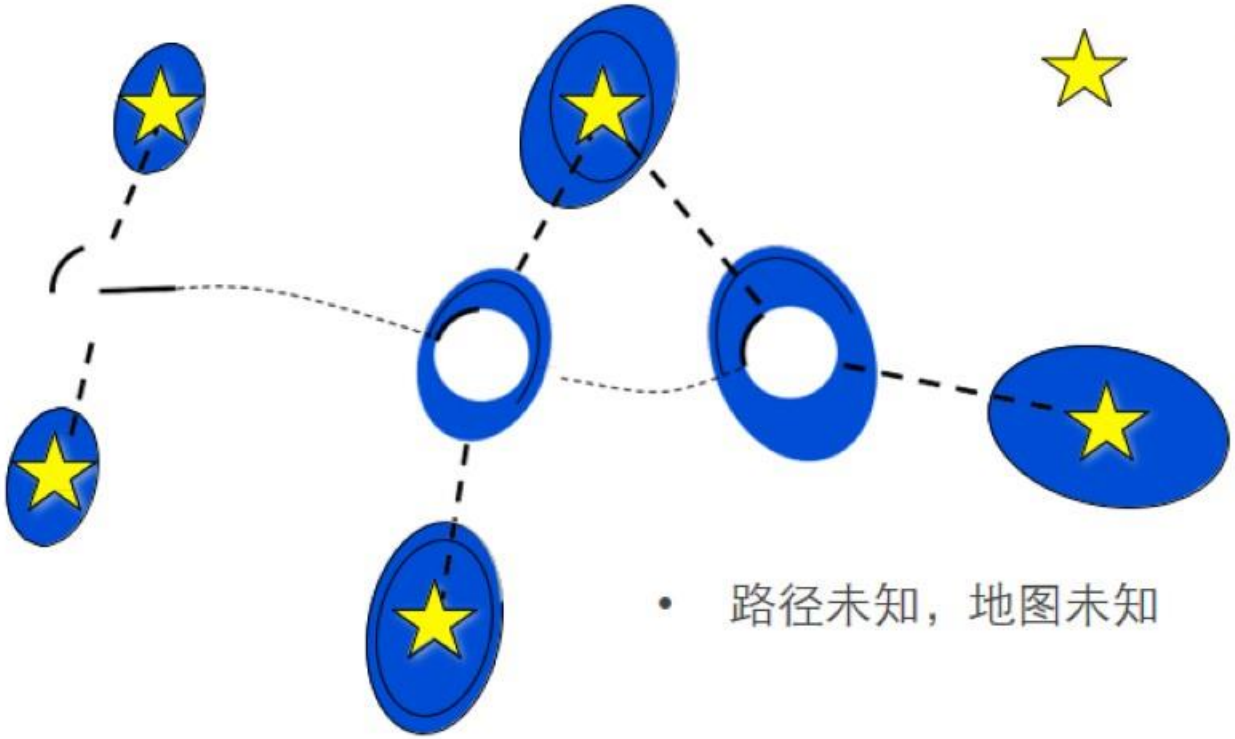
Point Cloud Library

➢ SLAM基础框架

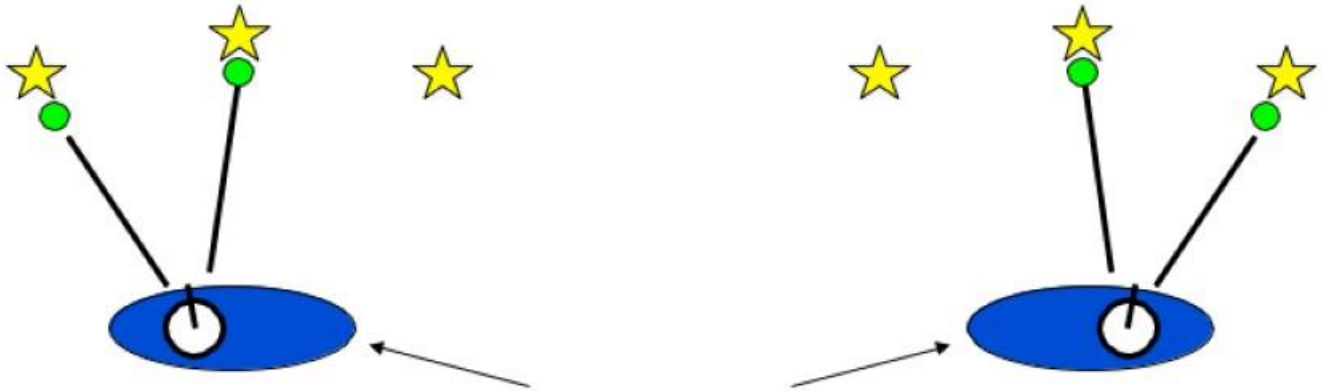
SLAM问题:

- 同步定位与建图“鸡生蛋，蛋生鸡”问题
- 定位需要地图
- 建图需要定位





- 路径未知，地图未知



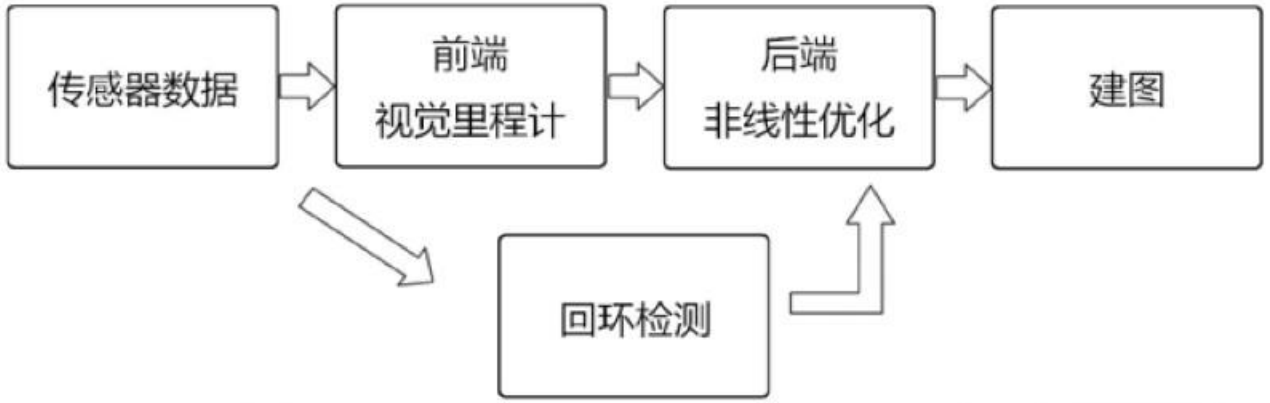
- 相对位姿未知，路标匹配未知

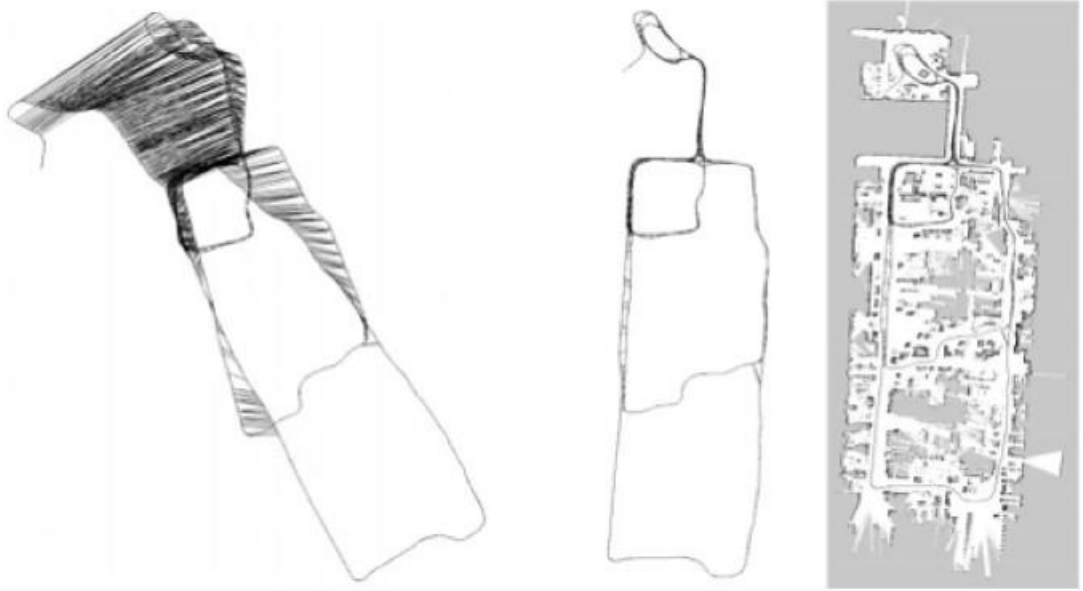
Robot Mapping, Crill Stachniss, freiburg

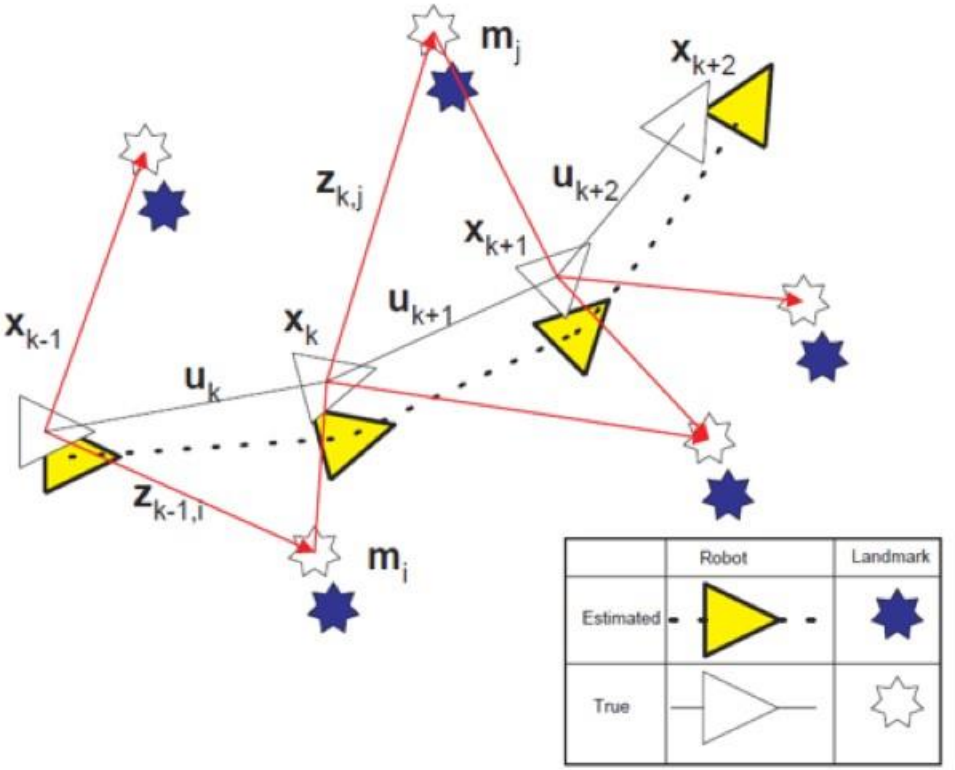
Point Cloud Library

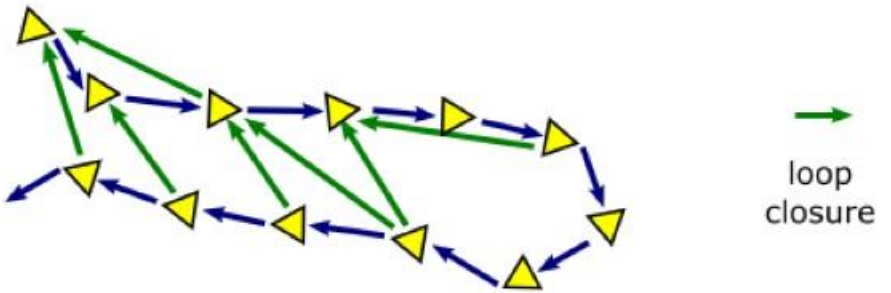
➢ SLAM基础框架

SLAM流程图:







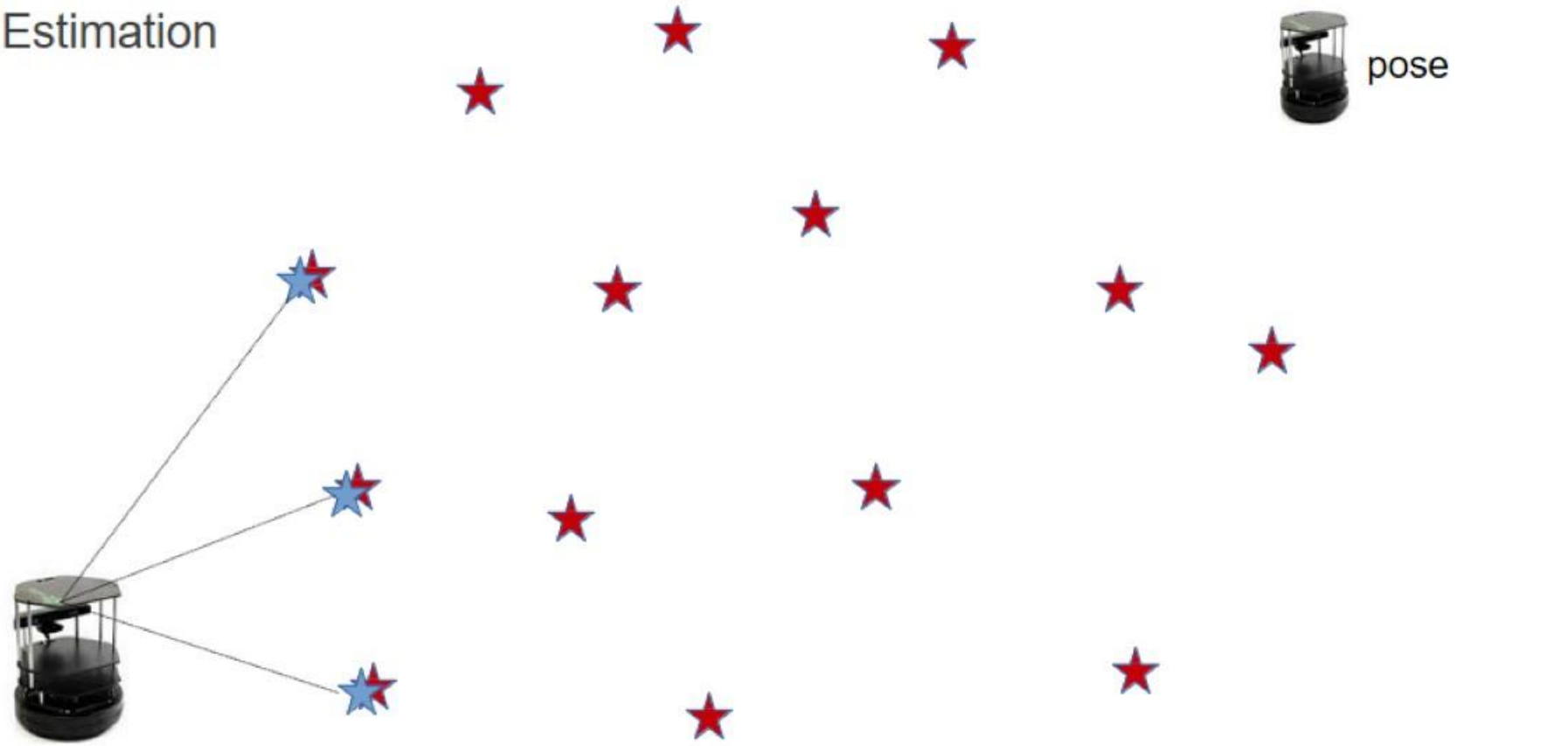


Point Cloud Library

- SLAM基础框架

SLAM问题:

- SLAM as Estimation

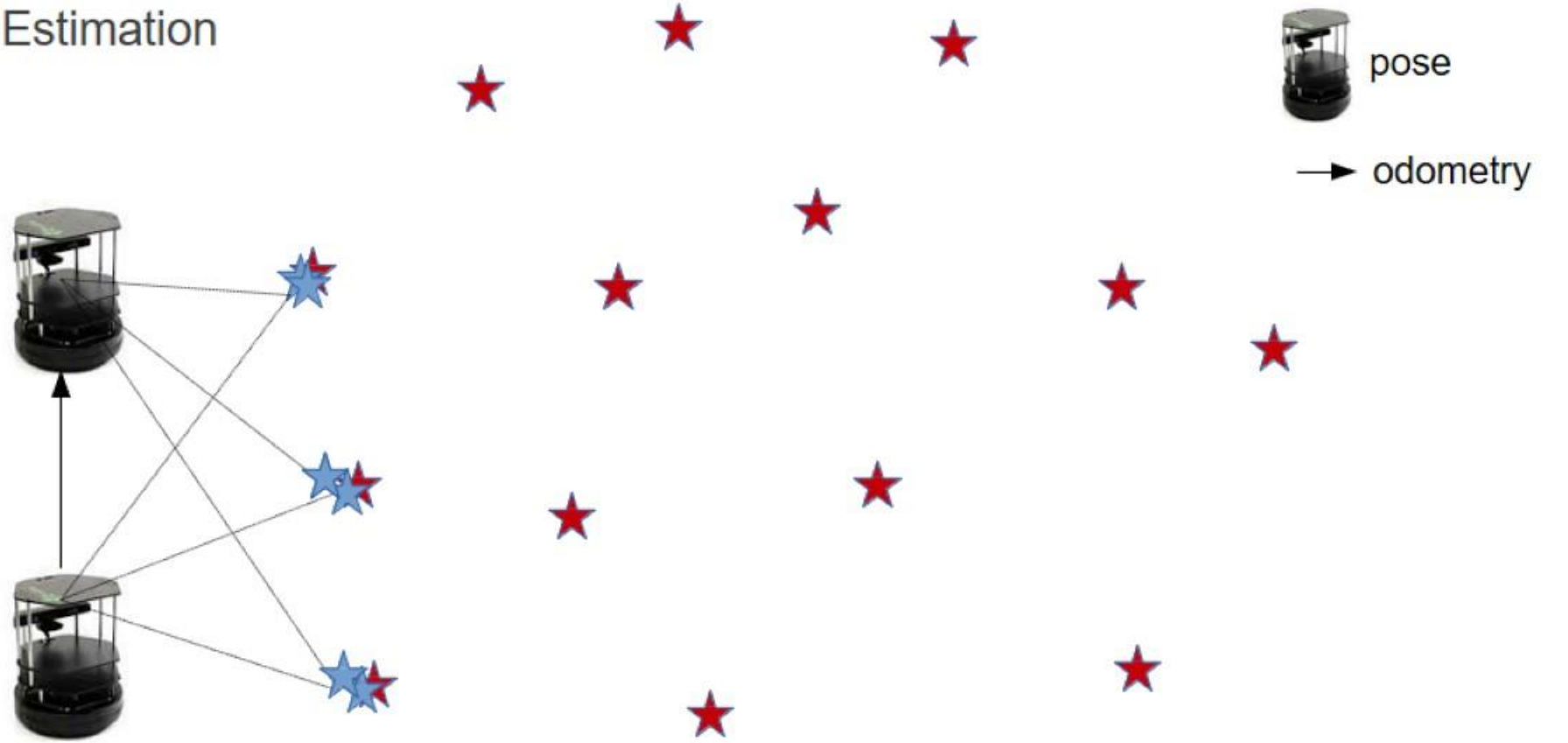


Point Cloud Library

➢ SLAM基础框架

SLAM问题:

- SLAM as Estimation

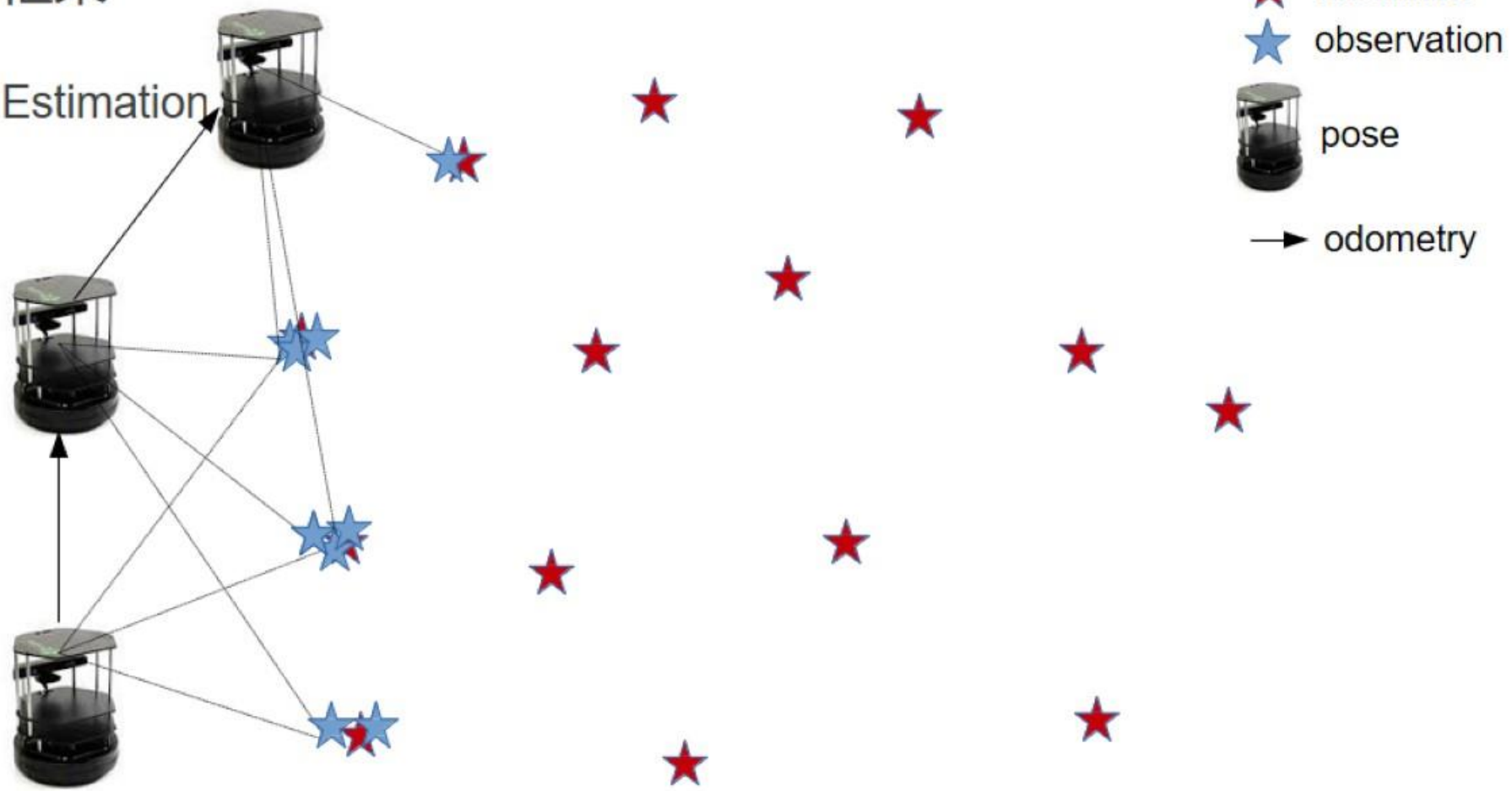


Point Cloud Library

- SLAM基础框架

SLAM问题:

- SLAM as Estimation



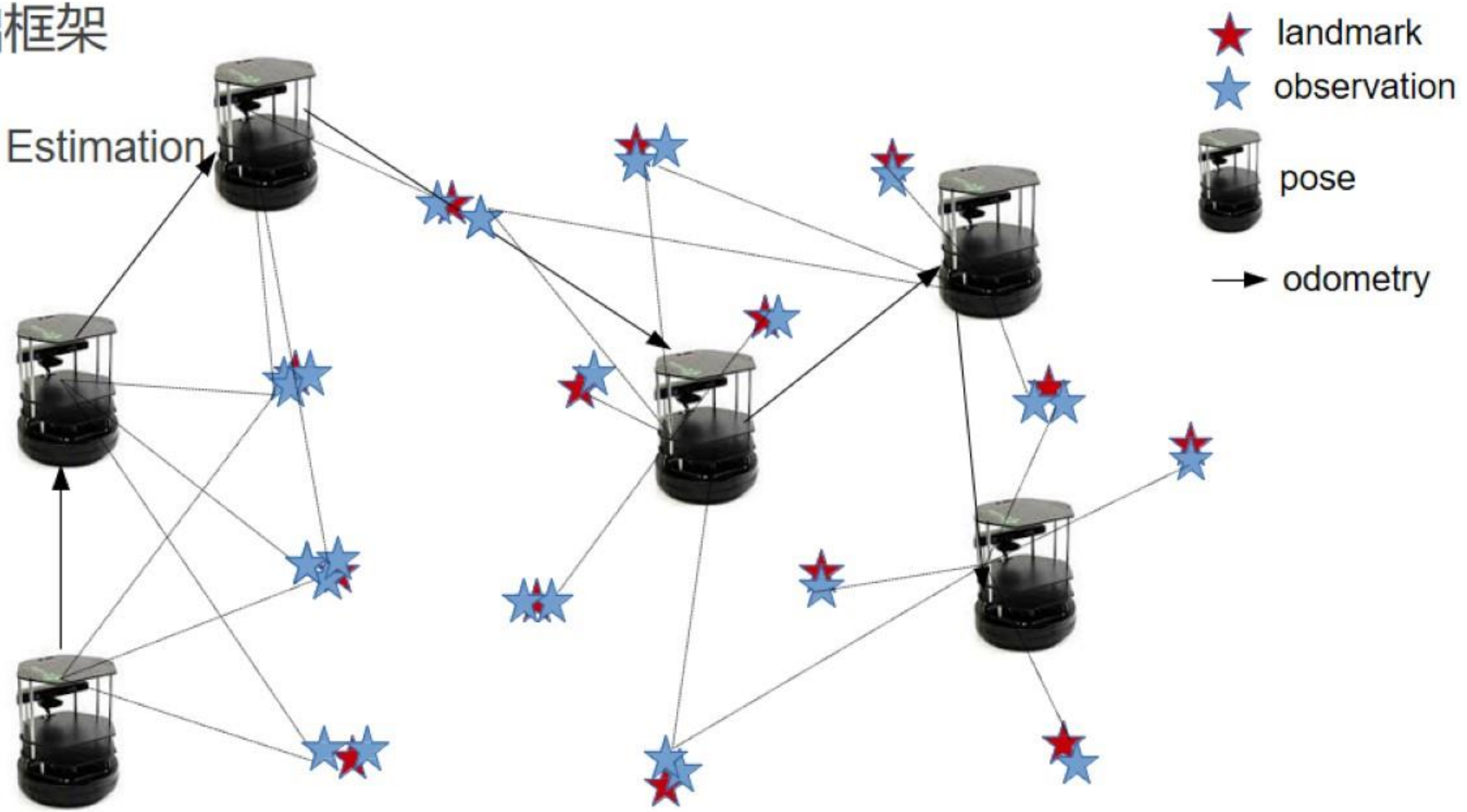
Robot Mapping, Crill Stachniss, freiburg

Point Cloud Library

- SLAM基础框架

SLAM问题:

- SLAM as Estimation



Robot Mapping, Crill Stachniss, freiburg

Point Cloud Library

- SLAM基础框架

SLAM问题:

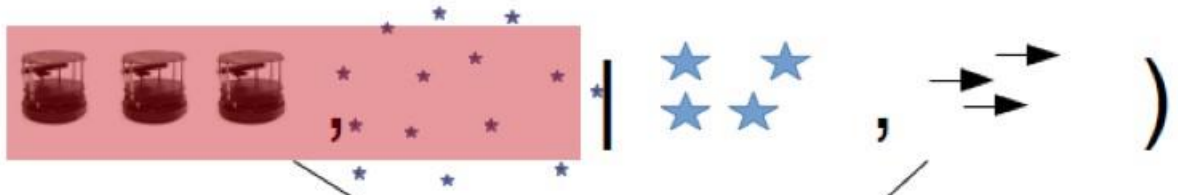
- SLAM as Estimation

Robot Mapping, Crill Stachniss, freiburg

➢ SLAM基础框架

SLAM问题:

- SLAM as Estimation



$$\mathbf{x}^* = \underset{\mathbf{x}}{\operatorname{argmax}}\, p(\mathbf{x}|\mathbf{z})$$

State状态

Observations观测

- 机器人控制量

$$u_{1:T} = \{u_1, u_2, u_3 \ldots, u_T\}$$

- 观测量

$$z_{1:T} = \{z_1, z_2, z_3 \ldots, z_T\}$$

- 地图

$$m$$

- 机器人路径（状态）

$$x_{0:T} = \{x_0, x_1, x_2 \ldots, x_T\}$$

$$p(x_{0:T}, m \mid z_{1:T}, u_{1:T})$$

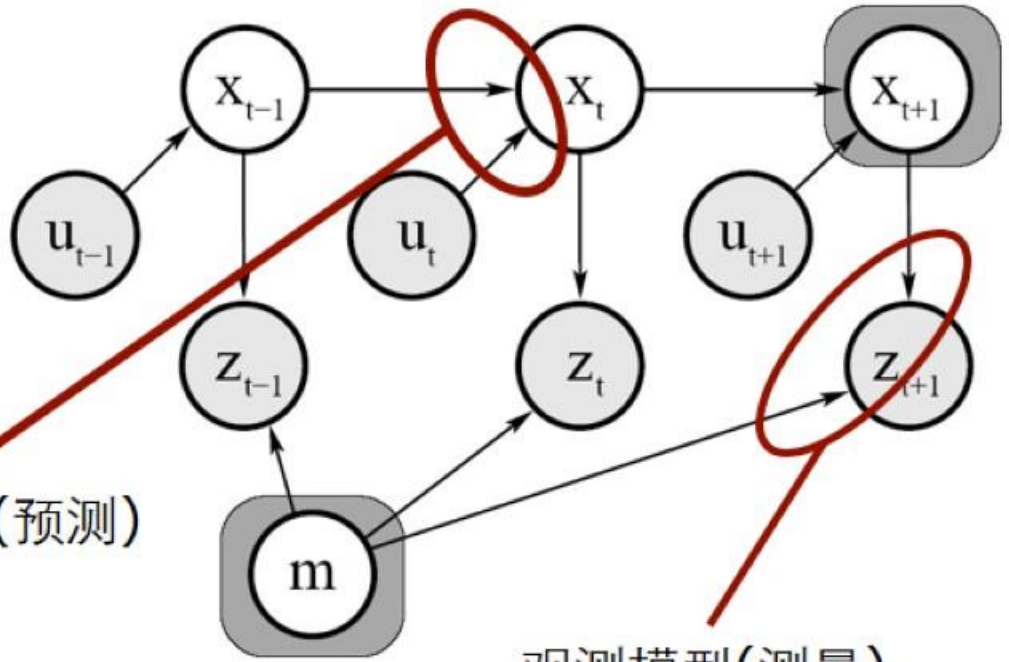
distribution path Map given observations controls

Point Cloud Library

➢ SLAM基础框架

SLAM问题:

- SLAM 最大后验估计->最大似然估计



- 运动方程

$$x_k = f(x_{k-1}, u_k) + w_k$$

- 观测方程

$$z_{k,j} = h(y_j, x_k) + v_{k,j}$$

$$\begin{aligned} \mathbf{x}^* &= \operatorname*{argmax}_{\mathbf{x}} p(\mathbf{x}|\mathbf{z}) \\ p(\mathbf{x}|\mathbf{z}) &= \frac{p(\mathbf{z}|\mathbf{x})p(\mathbf{x})}{p(\mathbf{z})} \\ &\propto p(\mathbf{z}|\mathbf{x}) \\ &= \prod_i p(\mathbf{z}_i|\mathbf{x}) \end{aligned}$$

SLAM基础框架

SLAM问题:

- 高斯假设

$$
\begin{aligned}
p(\mathbf{z}_i|\mathbf{x}) &= \mathcal{N}(\mathbf{z}_i; \mathbf{h}_i(\mathbf{x}), \mathbf{\Sigma}_i) \\
&\propto \exp\left(-(\underbrace{\mathbf{h}_i(\mathbf{x}) - \mathbf{z}_i}_{\mathbf{e}_i(\mathbf{x})})^T \underbrace{\mathbf{\Sigma}^{-1}}_{\mathbf{\Omega}_i} (\mathbf{h}_i(\mathbf{x}) - \mathbf{z}_i) \right)
\end{aligned}
$$

SLAM基础框架

SLAM问题:

- 高斯假设

Measurement (观测)

Prediction(预测)

state

$$p(\mathbf{z}_i|\mathbf{x}) = \mathcal{N}(\mathbf{z}_i; \mathbf{h}_i(\mathbf{x}), \mathbf{\Sigma}_i)$$

$$\propto \exp\left(-\underbrace{(\mathbf{h}_i(\mathbf{x}) - \mathbf{z}_i)}_{\mathbf{e}_i(\mathbf{x})}{}^T \underbrace{\mathbf{\Sigma}^{-1}}_{\mathbf{\Omega}_i} (\mathbf{h}_i(\mathbf{x}) - \mathbf{z}_i)\right)$$

error function

Point Cloud Library

➢ SLAM基础框架

SLAM问题:

- 最大似然估计->最小二乘求解

$$\begin{aligned}\mathbf{x}^* &= \operatorname*{argmax}_x \prod_i p(\mathbf{z}_i|\mathbf{x}) \\ &= \operatorname*{argmax}_x \prod_i \exp(-\mathbf{e}_i(\mathbf{x})^T \mathbf{\Omega}_i \mathbf{e}_i(\mathbf{x})) \\ &= \operatorname*{argmin}_x \sum_i \mathbf{e}_i(\mathbf{x})^T \mathbf{\Omega}_i \mathbf{e}_i(\mathbf{x})\end{aligned}$$

非线性最小二乘常用求解方法:

- 梯度下降法:
- 牛顿法:
- 高斯牛顿法:
- LM法(列文伯格-马夸尔特)

Point Cloud Library

- SLAM基础框架

SLAM分类:

- EKF滤波器
- 粒子滤波器
- 图优化

Extended_Kalman_filter$(\mu_{t-1}, \Sigma_{t-1}, u_t, z_t)$:

$\bar{\mu}_t = g(u_t, \mu_{t-1})$

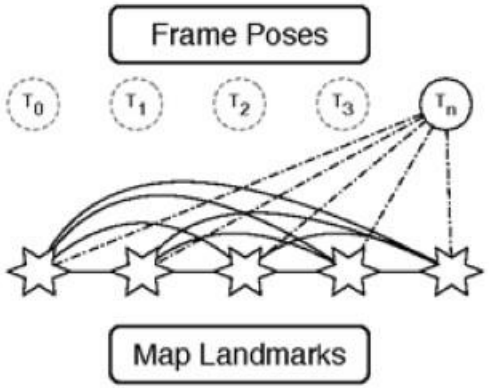
$\bar{\Sigma}_t = G_t \, \Sigma_{t-1} \, G_t^T + R_t$

$K_t = \bar{\Sigma}_t \, H_t^T (H_t \, \bar{\Sigma}_t \, H_t^T + Q_t)^{-1}$

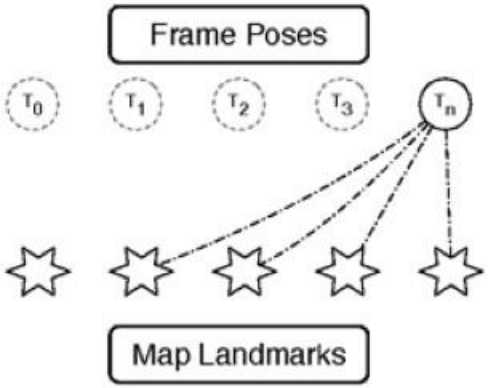
$\mu_t = \bar{\mu}_t + K_t(z_t - h(\bar{\mu}_t))$

$\Sigma_t = (I - K_t \, H_t) \, \bar{\Sigma}_t$

return μ_t, Σ_t



(a) Filter based systems



(b) Non-filter based systems

$$x_t = (\underbrace{x, y, \theta}_{\text{robot's pose}}, \underbrace{m_{1,x}, m_{1,y}}_{\text{landmark 1}}, \ldots, \underbrace{m_{n,x}, m_{n,y}}_{\text{landmark n}})^T$$

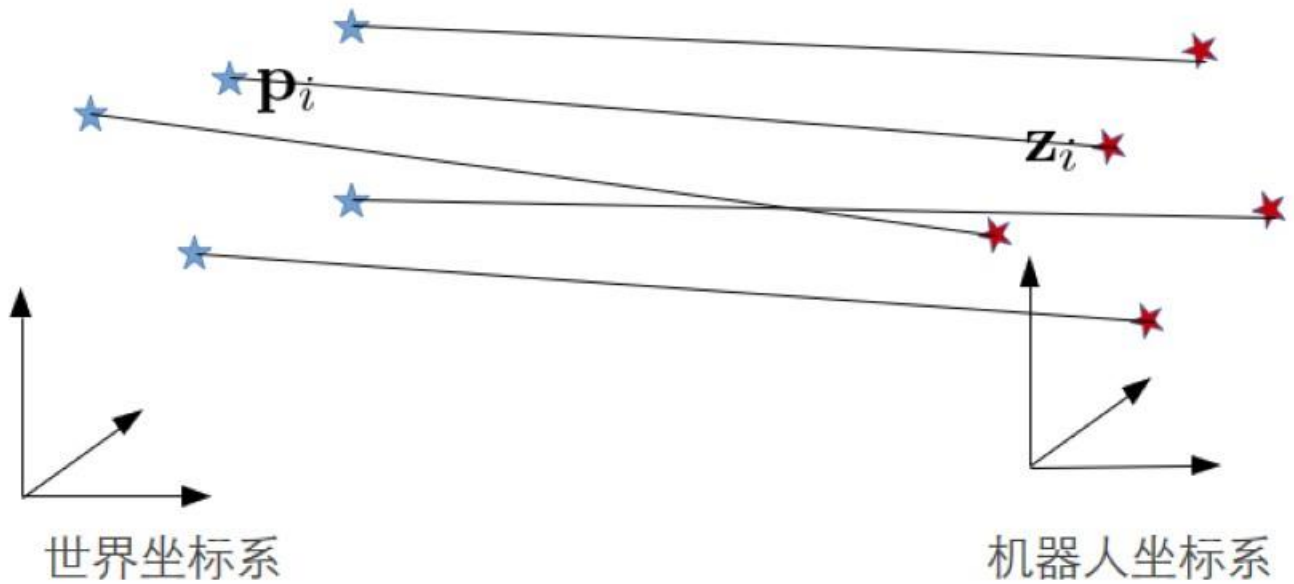
$$\underbrace{\begin{pmatrix} x \\ y \\ \theta \\ m_{1,x} \\ m_{1,y} \\ \vdots \\ m_{n,x} \\ m_{n,y} \end{pmatrix}}_{\mu} \quad \underbrace{\begin{pmatrix} \sigma_{xx} & \sigma_{xy} & \sigma_{x\theta} & \sigma_{xm_{1,x}} & \sigma_{xm_{1,y}} & \cdots & \sigma_{xm_{n,x}} & \sigma_{xm_{n,y}} \\ \sigma_{yx} & \sigma_{yy} & \sigma_{y\theta} & \sigma_{ym_{1,x}} & \sigma_{ym_{1,y}} & \cdots & \sigma_{m_{n,x}} & \sigma_{m_{n,y}} \\ \sigma_{\theta x} & \sigma_{\theta y} & \sigma_{\theta\theta} & \sigma_{\theta m_{1,x}} & \sigma_{\theta m_{1,y}} & \cdots & \sigma_{\theta m_{n,x}} & \sigma_{\theta m_{n,y}} \\ \sigma_{m_{1,x}x} & \sigma_{m_{1,x}y} & \sigma_{\theta} & \sigma_{m_{1,x}m_{1,x}} & \sigma_{m_{1,x}m_{1,y}} & \cdots & \sigma_{m_{1,x}m_{n,x}} & \sigma_{m_{1,x}m_{n,y}} \\ \sigma_{m_{1,y}x} & \sigma_{m_{1,y}y} & \sigma_{\theta} & \sigma_{m_{1,y}m_{1,x}} & \sigma_{m_{1,y}m_{1,y}} & \cdots & \sigma_{m_{1,y}m_{n,x}} & \sigma_{m_{1,y}m_{n,y}} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ \sigma_{m_{n,x}x} & \sigma_{m_{n,x}y} & \sigma_{\theta} & \sigma_{m_{n,x}m_{1,x}} & \sigma_{m_{n,x}m_{1,y}} & \cdots & \sigma_{m_{n,x}m_{n,x}} & \sigma_{m_{n,x}m_{n,y}} \\ \sigma_{m_{n,y}x} & \sigma_{m_{n,y}y} & \sigma_{\theta} & \sigma_{m_{n,y}m_{1,x}} & \sigma_{m_{n,y}m_{1,y}} & \cdots & \sigma_{m_{n,y}m_{n,x}} & \sigma_{m_{n,y}m_{n,y}} \end{pmatrix}}_{\Sigma}$$

Point Cloud Library

➢ SLAM基础框架

SLAM估计:

- icp slam建模

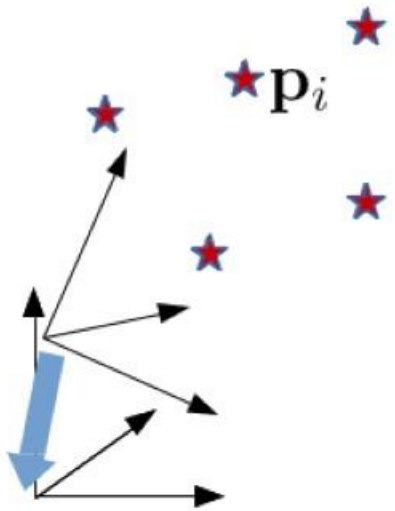


Point Cloud Library

➢ SLAM基础框架

SLAM估计:

- icp slam建模



世界坐标系

状态:

$$\mathbf{x} \in SE(3)$$

$$\mathbf{x} = (\underbrace{x\ y\ z}_{\mathbf{t}}\ \underbrace{\alpha_x\ \alpha_y\ \alpha_z}_{\alpha})^T$$

观测:

$$\mathbf{z} \in \Re^3$$

$$\mathbf{h}_i(\mathbf{x}) = \mathbf{R}(\alpha)\mathbf{p}_i + \mathbf{t}$$

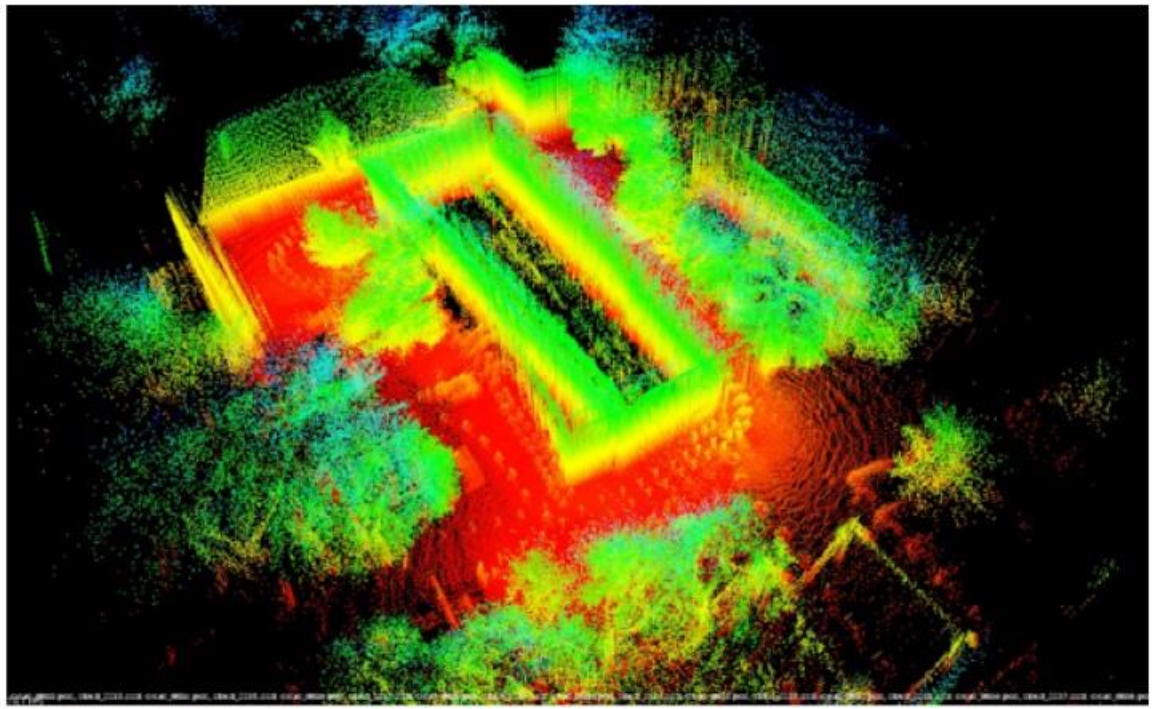
内容概要

点云配准方法
SLAM基础框架
帧间匹配与激光里程计
SLAM图优化基础

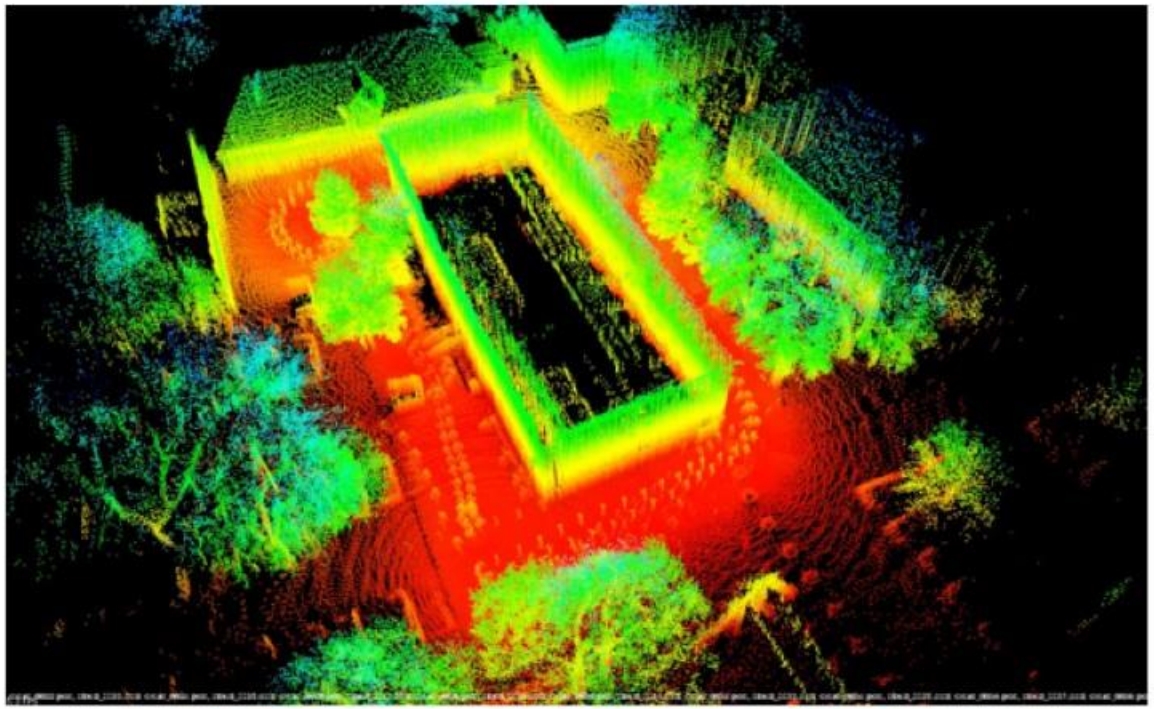
帧间匹配与激光里程计

- 帧间匹配

  Lidar SLAM前端：



Input Data



pcl_icp -d 0.75 -r 0.75 -i 50 ../data/cloud_0{022..130}.pcd

Jochen Sprickerhof:PCL::Registration

帧间匹配与激光里程计

帧间匹配

ICP变体：

- 给定原始点云与目标点云

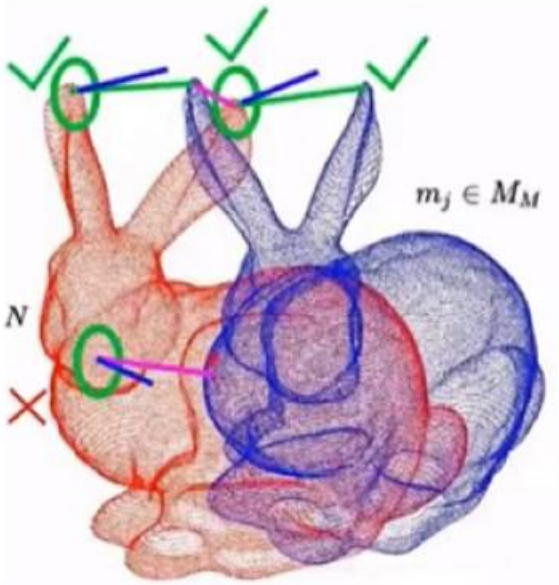
- Point-to-Point

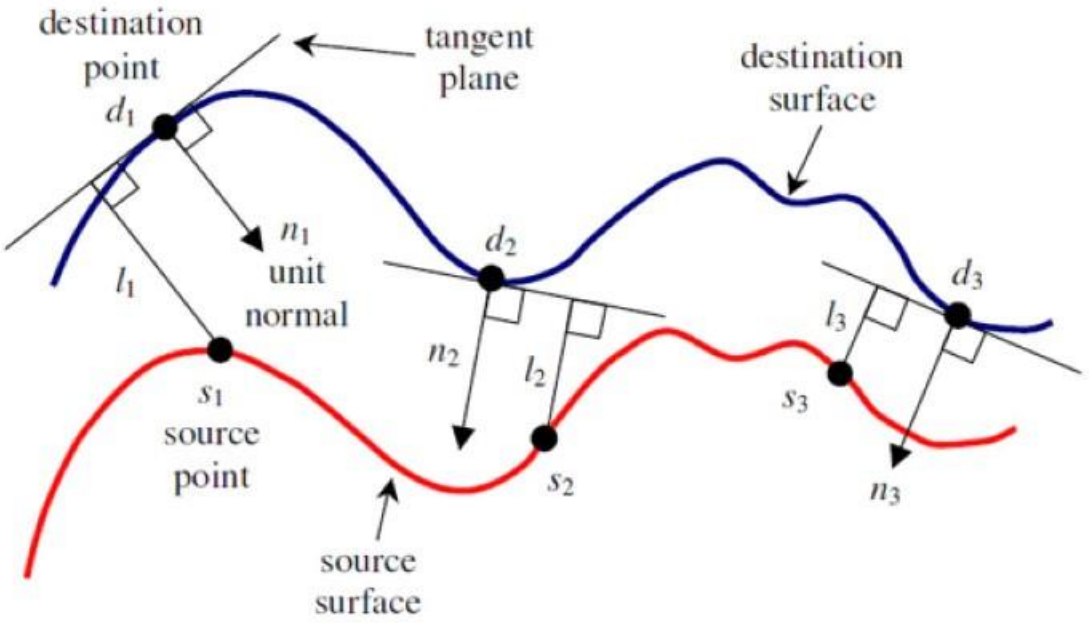
$$T \leftarrow \underset{T}{\operatorname{argmin}} \{\sum_i w_i ||T \cdot b_i - m_i||^2\};$$

- Point-to-Plane

$$T \leftarrow \underset{T}{\operatorname{argmin}} \{\sum_i w_i ||\eta_i \cdot (T \cdot b_i - m_i)||^2\}$$

- Plane-to-Plane





Jochen Sprickerhof:PCL::Registration

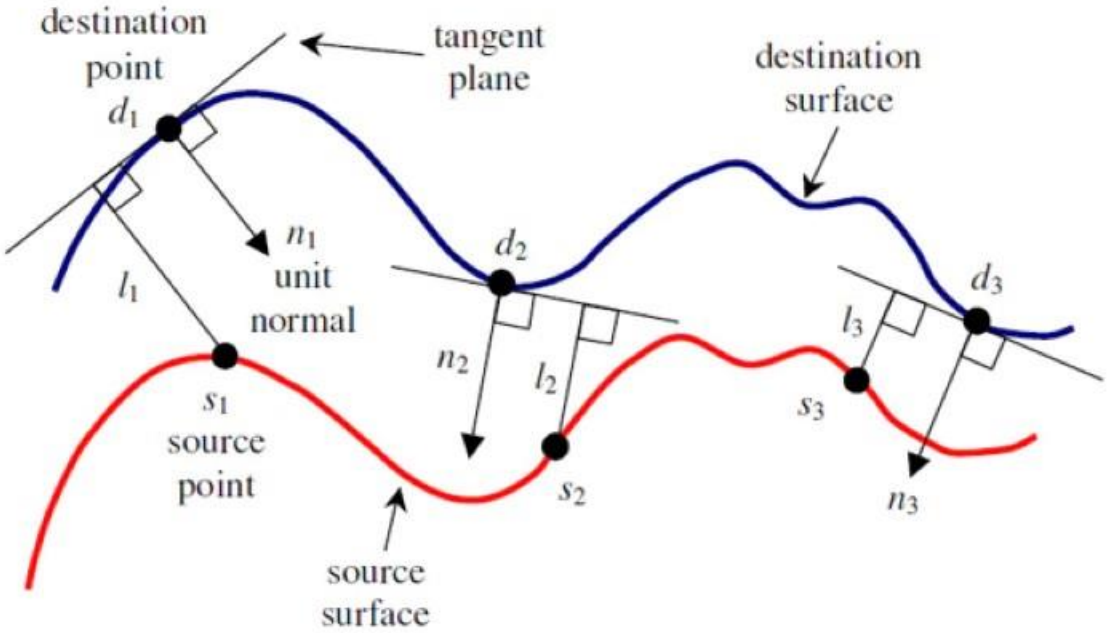
帧间匹配与激光里程计

➢ 帧间匹配

Point-to-Plane ICP:

$$\mathbf{M}_{\text{opt}} = \arg\min_{\mathbf{M}} \sum_i \left((\mathbf{M} \cdot \mathbf{s}_i - \mathbf{d}_i) \bullet \mathbf{n}_i \right)^2$$

$$\mathbf{M} = \mathbf{T}(t_x, t_y, t_z) \cdot \mathbf{R}(\alpha, \beta, \gamma)$$



$$\mathbf{T}(t_x, t_y, t_z) = \begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$\mathbf{R}(\alpha, \beta, \gamma) = \mathbf{R}_z(\gamma) \cdot \mathbf{R}_y(\beta) \cdot \mathbf{R}_x(\alpha) = \begin{pmatrix} r_{11} & r_{12} & r_{13} & 0 \\ r_{21} & r_{22} & r_{23} & 0 \\ r_{31} & r_{32} & r_{33} & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$\begin{aligned}
r_{11} &= \cos\gamma\cos\beta, \\
r_{12} &= -\sin\gamma\cos\alpha + \cos\gamma\sin\beta\sin\alpha, \\
r_{13} &= \sin\gamma\sin\alpha + \cos\gamma\sin\beta\cos\alpha, \\
r_{21} &= \sin\gamma\cos\beta, \\
r_{22} &= \cos\gamma\cos\alpha + \sin\gamma\sin\beta\sin\alpha, \\
r_{23} &= -\cos\gamma\sin\alpha + \sin\gamma\sin\beta\cos\alpha, \\
r_{31} &= -\sin\beta, \\
r_{32} &= \cos\beta\sin\alpha, \\
r_{33} &= \cos\beta\cos\alpha.
\end{aligned}$$

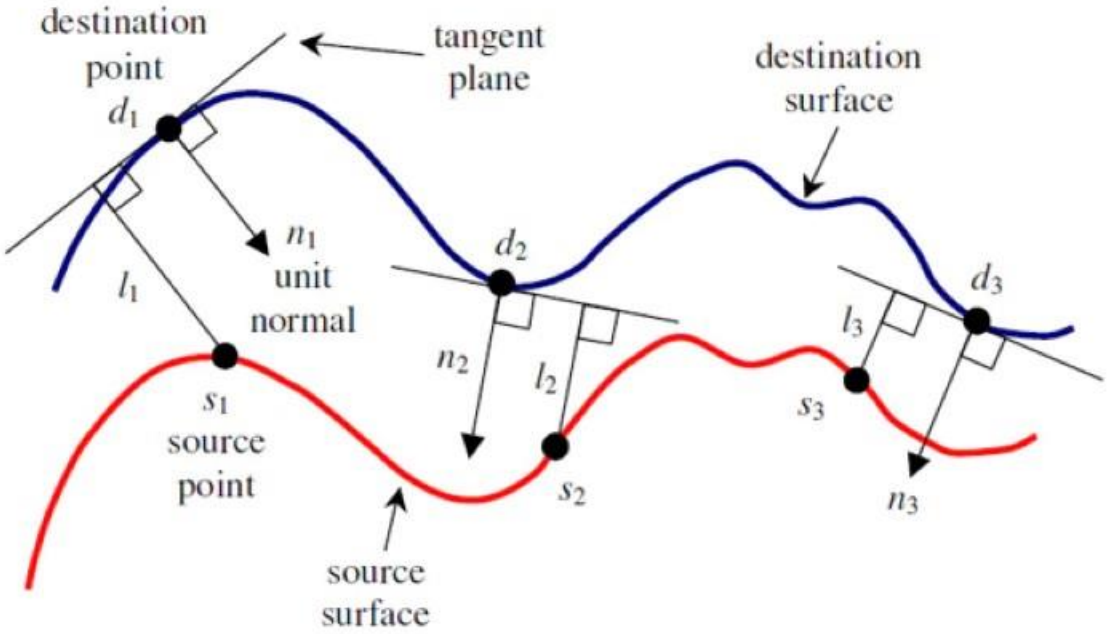
Linear Least-Squares Optimization for Point-to-Plane ICP Surface Registration

帧间匹配与激光里程计

帧间匹配

Point-to-Plane ICP:

$$\mathbf{M}_{\text{opt}} = \arg\min_{\mathbf{M}} \sum_i \left((\mathbf{M} \cdot \mathbf{s}_i - \mathbf{d}_i) \bullet \mathbf{n}_i \right)^2$$



$$\mathbf{R}(\alpha,\beta,\gamma) \approx \begin{pmatrix} 1 & \alpha\beta-\gamma & \alpha\gamma+\beta & 0 \\ \gamma & \alpha\beta\gamma+1 & \beta\gamma-\alpha & 0 \\ -\beta & \alpha & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$\approx \begin{pmatrix} 1 & -\gamma & \beta & 0 \\ \gamma & 1 & -\alpha & 0 \\ -\beta & \alpha & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} = \hat{\mathbf{R}}(\alpha,\beta,\gamma)$$

$$(\hat{\mathbf{M}} \cdot \mathbf{s}_i - \mathbf{d}_i) \bullet \mathbf{n}_i = \left(\hat{\mathbf{M}} \cdot \begin{pmatrix} s_{ix} \\ s_{iy} \\ s_{iz} \\ 1 \end{pmatrix} - \begin{pmatrix} d_{ix} \\ d_{iy} \\ d_{iz} \\ 1 \end{pmatrix} \right) \bullet \begin{pmatrix} n_{ix} \\ n_{iy} \\ n_{iz} \\ 0 \end{pmatrix}$$

$$= [(n_{iz}s_{iy} - n_{iy}s_{iz})\alpha + (n_{ix}s_{iz} - n_{iz}s_{ix})\beta + (n_{iy}s_{ix} - n_{ix}s_{iy})\gamma + n_{ix}t_x + n_{iy}t_y + n_{iz}t_z] -$$

$$[n_{ix}d_{ix} + n_{iy}d_{iy} + n_{iz}d_{iz} - n_{ix}s_{ix} - n_{iy}s_{iy} - n_{iz}s_{iz}].$$

Linear Least-Squares Optimization for Point-to-Plane ICP Surface Registration

帧间匹配与激光里程计

➢ 帧间匹配

Point-to-Plane ICP:

$$\mathbf{M}_{\text{opt}} = \arg\min_{\mathbf{M}} \sum_i \left((\mathbf{M}\cdot\mathbf{s}_i - \mathbf{d}_i)\bullet\mathbf{n}_i \right)^2$$

$$(\hat{\mathbf{M}}\cdot\mathbf{s}_i - \mathbf{d}_i)\bullet\mathbf{n}_i = \left(\hat{\mathbf{M}}\cdot\begin{pmatrix} s_{ix} \\ s_{iy} \\ s_{iz} \\ 1 \end{pmatrix} - \begin{pmatrix} d_{ix} \\ d_{iy} \\ d_{iz} \\ 1 \end{pmatrix} \right) \bullet \begin{pmatrix} n_{ix} \\ n_{iy} \\ n_{iz} \\ 0 \end{pmatrix}$$

$$= [(n_{iz}s_{iy} - n_{iy}s_{iz})\alpha + (n_{ix}s_{iz} - n_{iz}s_{ix})\beta + (n_{iy}s_{ix} - n_{ix}s_{iy})\gamma + n_{ix}t_x + n_{iy}t_y + n_{iz}t_z] -$$

$$[n_{ix}d_{ix} + n_{iy}d_{iy} + n_{iz}d_{iz} - n_{ix}s_{ix} - n_{iy}s_{iy} - n_{iz}s_{iz}].$$

$$\min_{\hat{\mathbf{M}}} \sum_i \left((\hat{\mathbf{M}}\cdot\mathbf{s}_i - \mathbf{d}_i)\bullet\mathbf{n}_i \right)^2 = \min_{\mathbf{x}} |\mathbf{A}\mathbf{x} - \mathbf{b}|^2$$

$$\mathbf{b} = \begin{pmatrix} n_{1x}d_{1x} + n_{1y}d_{1y} + n_{1z}d_{1z} - n_{1x}s_{1x} - n_{1y}s_{1y} - n_{1z}s_{1z} \\ n_{2x}d_{2x} + n_{2y}d_{2y} + n_{2z}d_{2z} - n_{2x}s_{2x} - n_{2y}s_{2y} - n_{2z}s_{2z} \\ \vdots \\ n_{Nx}d_{Nx} + n_{Ny}d_{Ny} + n_{Nz}d_{Nz} - n_{Nx}s_{Nx} - n_{Ny}s_{Ny} - n_{Nz}s_{Nz} \end{pmatrix}$$

$$\mathbf{x} = \begin{pmatrix} \alpha & \beta & \gamma & t_x & t_y & t_z \end{pmatrix}^{\mathrm{T}}$$

$$\mathbf{A} = \begin{pmatrix} a_{11} & a_{12} & a_{13} & n_{1x} & n_{1y} & n_{1z} \\ a_{21} & a_{22} & a_{23} & n_{2x} & n_{2y} & n_{2z} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ a_{N1} & a_{N2} & a_{N3} & n_{Nx} & n_{Ny} & n_{Nz} \end{pmatrix}$$

$$a_{i1} = n_{iz}s_{iy} - n_{iy}s_{iz},$$

$$a_{i2} = n_{ix}s_{iz} - n_{iz}s_{ix},$$

$$a_{i3} = n_{iy}s_{ix} - n_{ix}s_{iy}.$$

$$\mathbf{x}_{\text{opt}} = \arg\min_{\mathbf{x}} |\mathbf{A}\mathbf{x} - \mathbf{b}|^2$$

Linear Least-Squares Optimization for Point-to-Plane ICP Surface Registration

帧间匹配与激光里程计

➢ 帧间匹配

Point-to-Plane ICP:
IterativeClosestPointWithNormals

```
pcl::IterativeClosestPointWithNormals<pcl::PointXYZINormal,
pcl::PointXYZINormal>::Ptr
icp (new
pcl::IterativeClosestPointWithNormals<pcl::PointXYZINormal,pcl::PointXYZINormal>());
icp->setTransformationEpsilon(0.0000000001);
icp->setMaxCorrespondenceDistance(2.0);
icp->setMaximumIterations(50);
icp->setRANSACIterations(20);
icp->setInputSource(cloud_source_normals); //
icp->setInputTarget(cloud_target_normals);
icp->align(*cloud_source_trans_normals, guess); //
```

Jochen Sprickerhof:PCL::Registration

帧间匹配与激光里程计

➢ 帧间匹配

ICP变体：

- 给定原始点云与目标点云

- Point-to-Point

$$T \leftarrow \underset{T}{\operatorname{argmin}} \{\sum_i w_i ||T \cdot b_i - m_i||^2\};$$

- Point-to-Plane

$$T \leftarrow \underset{T}{\operatorname{argmin}} \{\sum_i w_i ||\eta_i \cdot (T \cdot b_i - m_i)||^2\}$$

- Plane-to-Plane

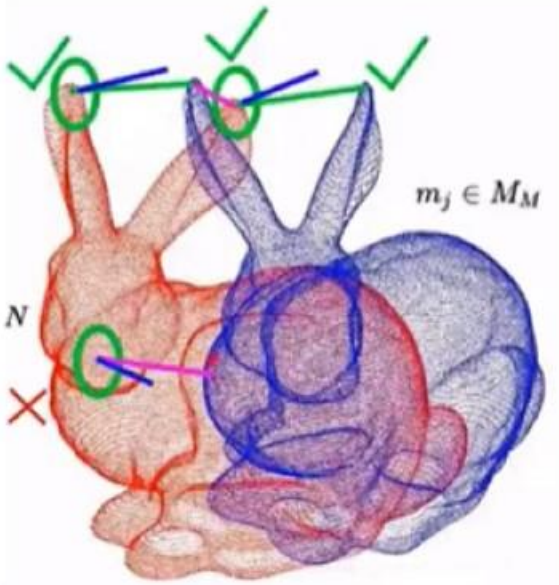
$$A = \{a_i\}_{i=1,\ldots,N} \quad \hat{A} = \{\hat{a}_i\} \quad a_i \sim \mathcal{N}(\hat{a}_i, C_i^A)$$

$$B = \{b_i\}_{i=1,\ldots,N} \quad \hat{B} = \{\hat{b}_i\} \quad b_i \sim \mathcal{N}(\hat{b}_i, C_i^B)$$

$$||m_i - T \cdot b_i|| > d_{max} \quad \hat{b}_i = \mathbf{T}^* \hat{a}_i$$

$$d_i^{(\mathbf{T})} = b_i - \mathbf{T}a_i \quad d_i^{(\mathbf{T}^*)} \quad d_i^{(\mathbf{T}^*)} \sim \mathcal{N}(\hat{b}_i - (\mathbf{T}^*)\hat{a}_i, C_i^B + (\mathbf{T}^*)C_i^A(\mathbf{T}^*)^T)$$

$$= \mathcal{N}(0, C_i^B + (\mathbf{T}^*)C_i^A(\mathbf{T}^*)^T)$$



Jochen Sprickerhof:PCL::Registration; Generalized-ICP

帧间匹配与激光里程计

➢ 帧间匹配

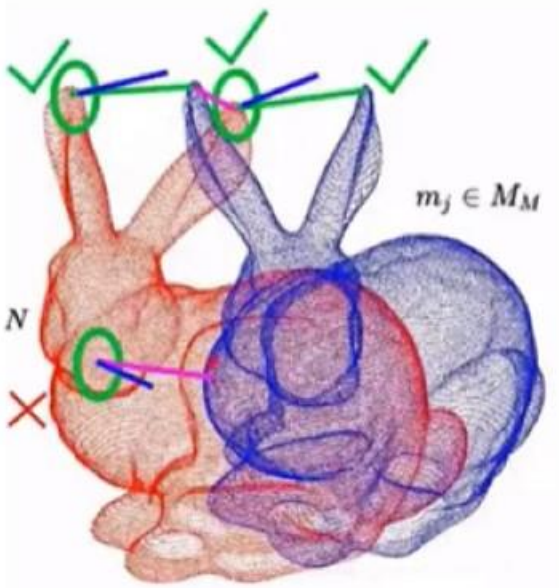
Plane-to-Plane:

- Generalized-ICP

$$C_i^B = \mathbf{R}_{\mu_i} \cdot \begin{pmatrix} \epsilon & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \cdot \mathbf{R}_{\mu_i}^T$$

$$C_i^A = \mathbf{R}_{\nu_i} \cdot \begin{pmatrix} \epsilon & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \cdot \mathbf{R}_{\nu_i}^T$$

- Plane-to-Plane



$$d_i^{(\mathbf{T}^*)} \sim \mathcal{N}(\hat{b}_i - (\mathbf{T}^*)\hat{a}_i, C_i^B + (\mathbf{T}^*)C_i^A(\mathbf{T}^*)^T)$$
$$= \mathcal{N}(0, C_i^B + (\mathbf{T}^*)C_i^A(\mathbf{T}^*)^T)$$

$$\mathbf{T} = \underset{\mathbf{T}}{\operatorname{argmax}} \prod_i p(d_i^{(\mathbf{T})}) = \underset{\mathbf{T}}{\operatorname{argmax}} \sum_i \log(p(d_i^{(\mathbf{T})}))$$

$$\mathbf{T} = \underset{\mathbf{T}}{\operatorname{argmin}} \sum_i {d_i^{(\mathbf{T})}}^T (C_i^B + \mathbf{T}C_i^A\mathbf{T}^T)^{-1} d_i^{(\mathbf{T})}$$

$C_i^B = I$ $\quad \mathbf{T} = \underset{\mathbf{T}}{\operatorname{argmin}} \sum_i {d_i^{(\mathbf{T})}}^T d_i^{(\mathbf{T})}$

$C_i^A = 0$

$$= \underset{\mathbf{T}}{\operatorname{argmin}} \sum_i ||d_i^{(\mathbf{T})}||^2$$

- Point-to-Point

$C_i^B = \mathbf{P_i}^{-1}$ $\quad \mathbf{T} = \underset{\mathbf{T}}{\operatorname{argmin}} \{\sum_i ||\mathbf{P_i} \cdot d_i||^2\}$

$C_i^A = 0$

$$||\mathbf{P_i} \cdot d_i||^2 = (\mathbf{P_i} \cdot d_i)^T \cdot (\mathbf{P_i} \cdot d_i)$$
$$= d_i^T \cdot \mathbf{P_i} \cdot d_i$$

- Point-to-Plane

Jochen Sprickerhof:PCL::Registration; Generalized-ICP

帧间匹配与激光里程计

➢ 帧间匹配

Plane-to-Plane ICP (Generalized-ICP):
GeneralizedIterativeClosestPoint

```
pcl::GeneralizedIterativeClosestPoint<pcl::PointXYZI, pcl::PointXYZI>icp;
// GICP 泛化的ICP，或者叫Plane to Plane ICP
icp.setTransformationEpsilon(0.0000000001);
icp.setMaxCorrespondenceDistance(2.0);
icp.setMaximumIterations(50);
icp.setRANSACIterations(20);
icp.setInputTarget(tar);
icp.setInputSource(src);
pcl::PointCloud<pcl::PointXYZI> unused_result;
icp.align(unused_result, guess);
```

Jochen Sprickerhof:PCL::Registration

帧间匹配与激光里程计

➢ 帧间匹配

NDT(Normal Distribution Transform):

- NDT算法的基本思想是构建多维变量的正态分布，如果变换参数是两个点云的最佳匹配，则变换点的概率密度最大。因此，用优化的方法求出使得概率密度之和最大的变换参数。

- 将target点云划分成指定大小（CellSize）的网格或体素（Voxel）；并计算每个网格的多维正态分布参数：均值和协方差矩阵 $\mathbf{q} = \frac{1}{n}\sum_i \mathbf{x}_i \quad \Sigma = \frac{1}{n}\sum_i (\mathbf{x_i} - \mathbf{q})(\mathbf{x_i} - \mathbf{q})^t$
- 初始化变换参数
- 对于要配准的点云（source scan），通过变换T将其转换到target点云的网格中
- 根据target的正态分布参数，计算当前帧每个转换点的概率密度（probability density function, PDF）

$$p(\mathbf{x}) \sim \exp(-\frac{(\mathbf{x}-\mathbf{q})^t \Sigma^{-1} (\mathbf{x}-\mathbf{q})}{2})$$

- NDT配准得分（score），每个点云可能会占据多个网格，通过对每个网格计算出的概率密度相加得总分

$$\mathtt{score}(\mathbf{p}) = \sum_i \exp(\frac{-(\mathbf{x'_i} - \mathbf{q_i})^{\mathbf{t}} \mathbf{\Sigma_i^{-1}} (\mathbf{x'_i} - \mathbf{q_i})}{2})$$

- 优化目标函数（score）牛顿优化算法

Jochen Sprickerhof:PCL::Registration

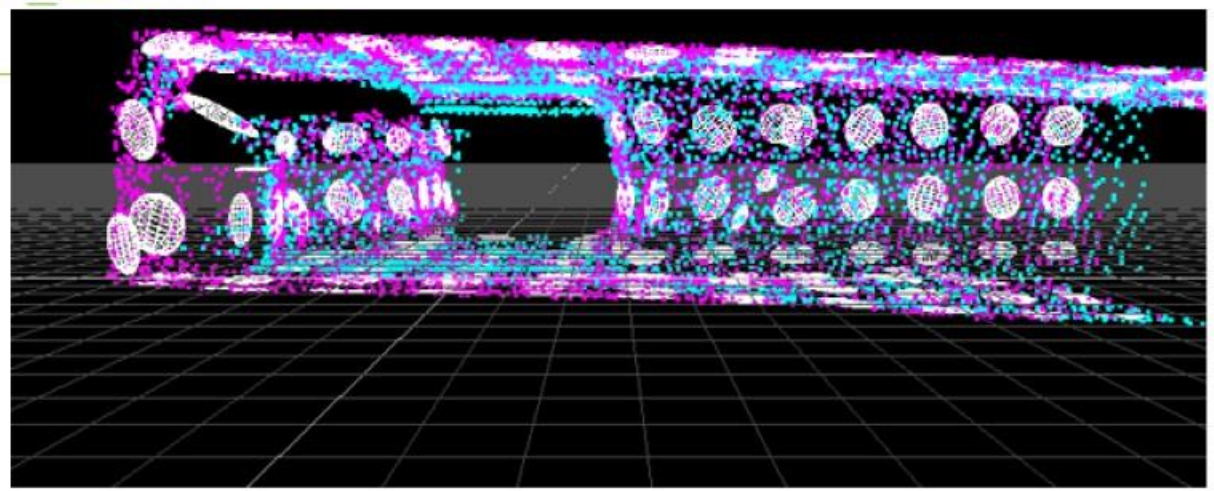
帧间匹配与激光里程计

➢ 帧间匹配

NDT(Normal Distribution Transform) :

NormalDistributionsTransform

```
pcl::NormalDistributionsTransform<pcl::PointXYZ, pcl::PointXYZ> ndt;
ndt.setTransformationEpsilon (0.01);
ndt.setStepSize (0.1); // for More-Thuente line search.
ndt.setResolution (1.0); //NDT grid structure (VoxelGridCovariance).
ndt.setMaximumIterations (35);
ndt.setInputCloud (filtered_cloud);
ndt.setInputTarget (target_cloud);
pcl::PointCloud<pcl::PointXYZ>::Ptr output_cloud;
ndt.align (*output_cloud, init_guess);
```



Jochen Sprickerhof:PCL::Registration

帧间匹配与激光里程计

- 帧间匹配

PCL SLAM API:

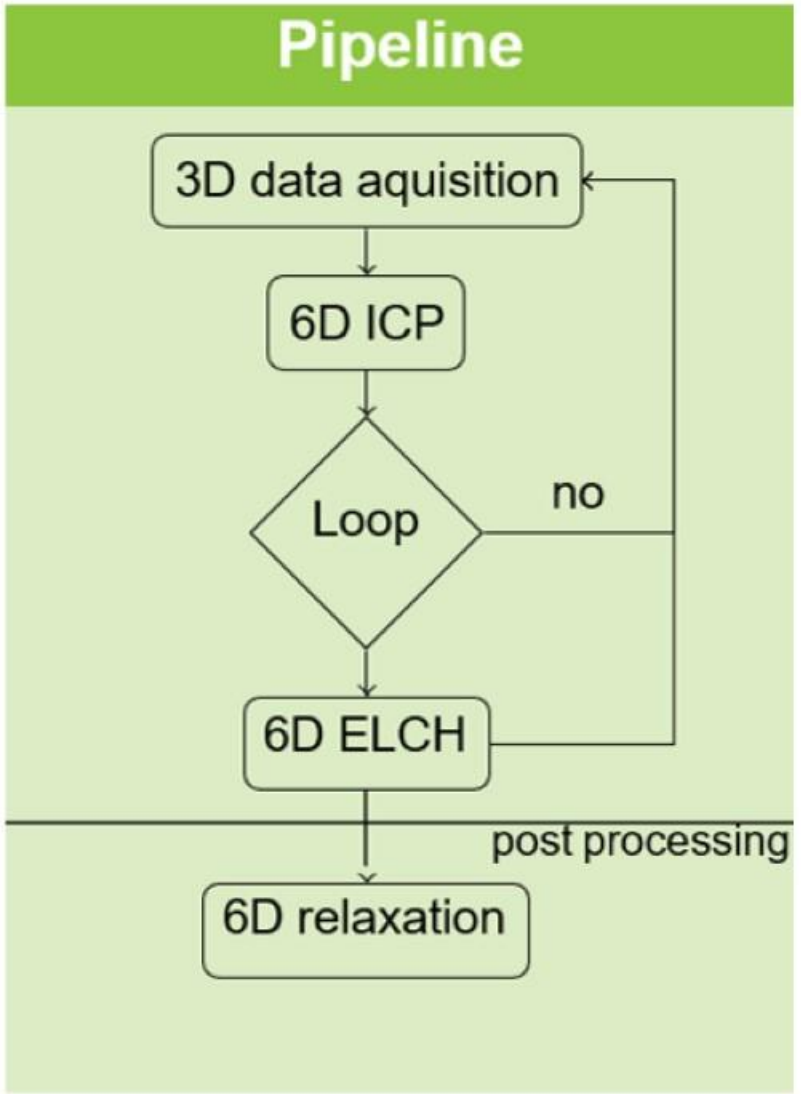
Pipeline:

前端匹配：6D ICP

闭环修正：6D ELCH

后处理：6D Lu&Milios(LUM)

G2O



Jochen Sprickerhof:PCL::Registration

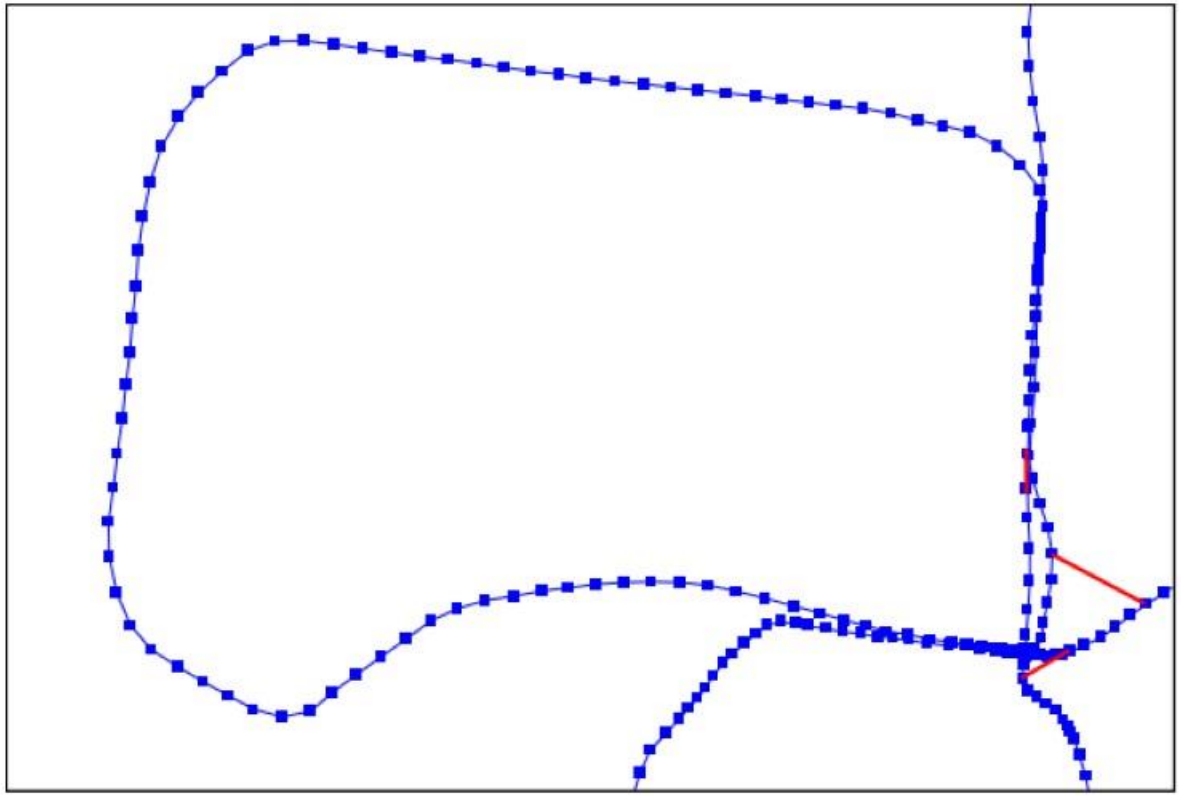
帧间匹配与激光里程计

➢ 帧间匹配

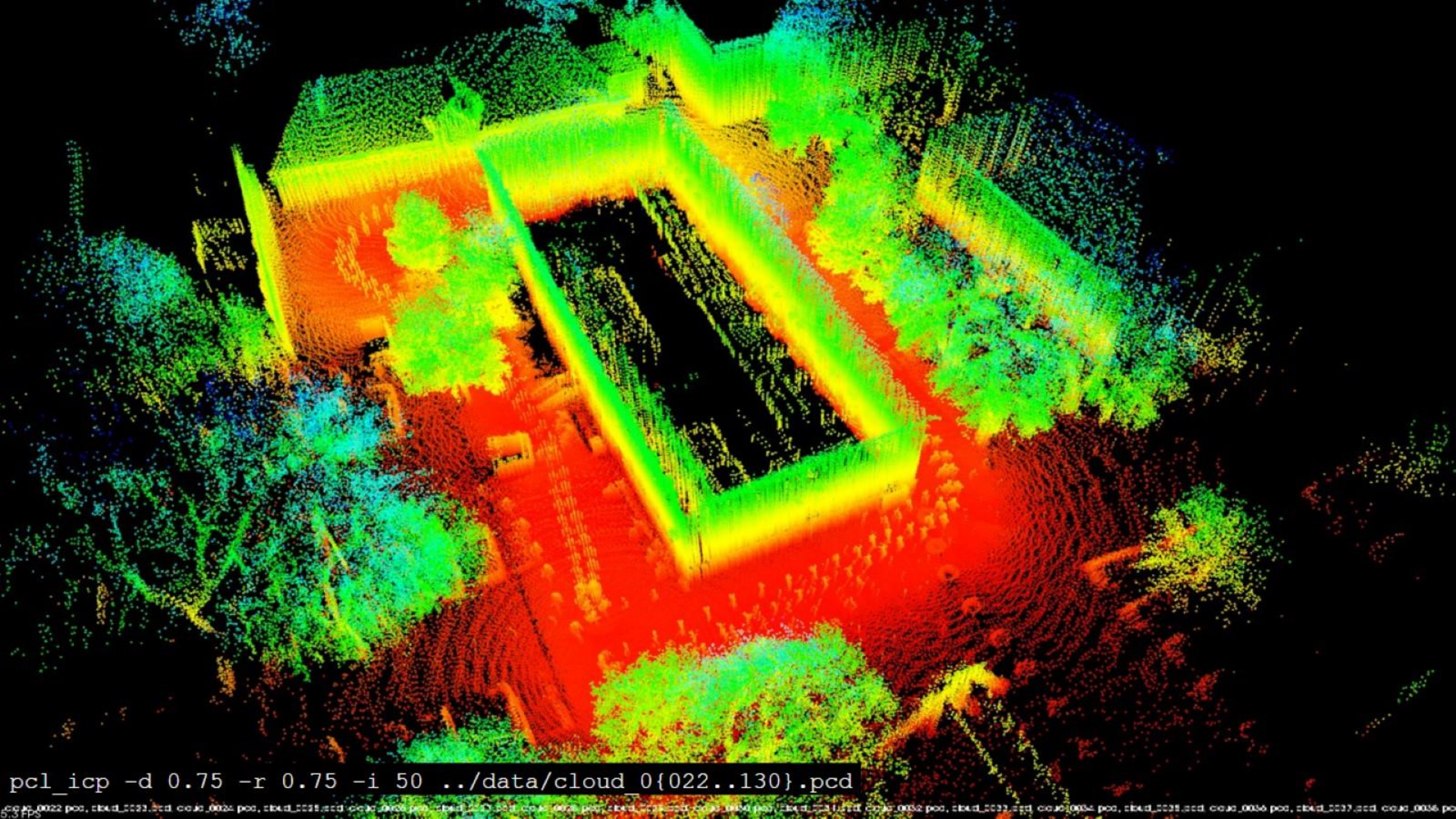
PCL SLAM API:

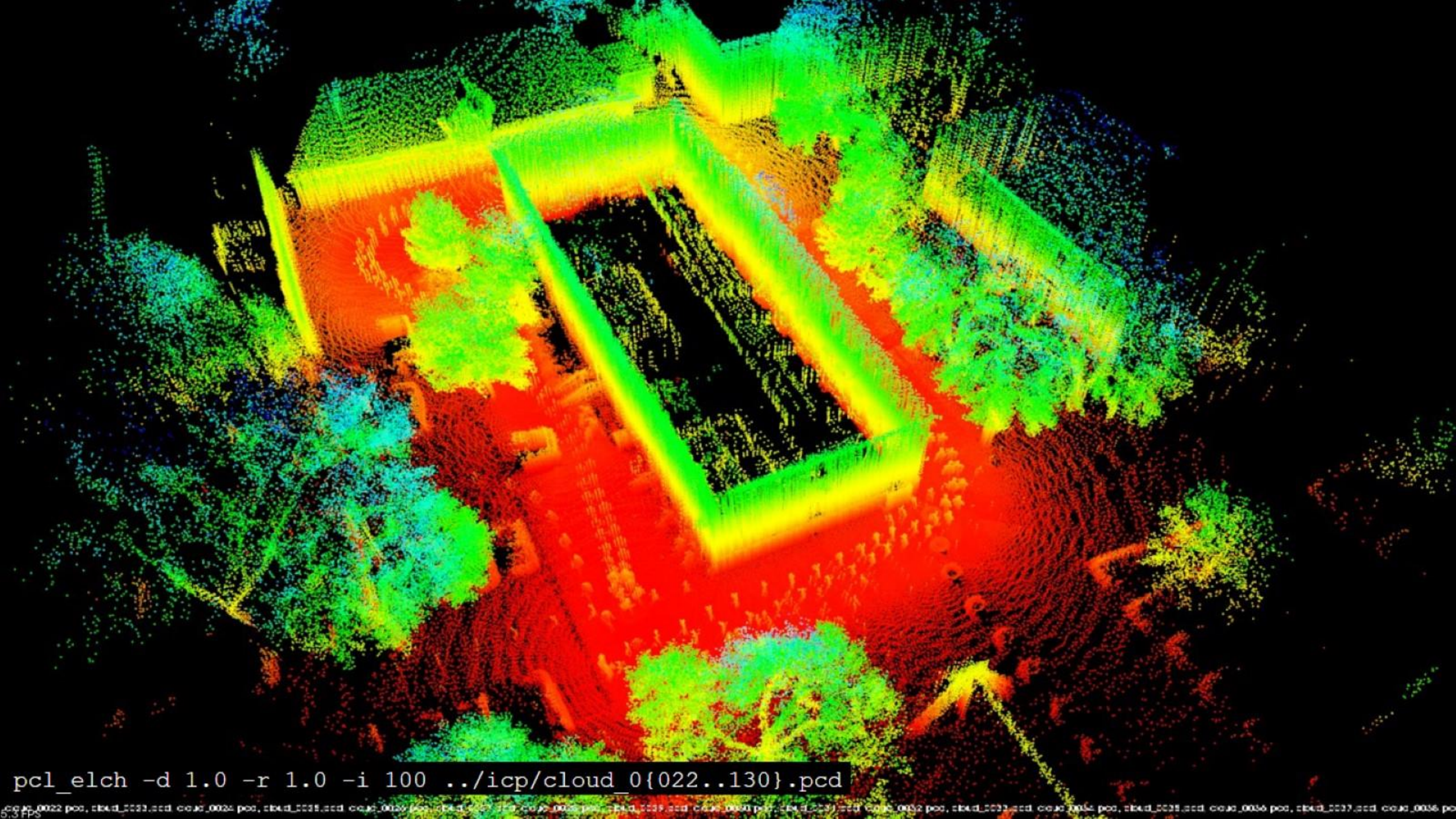
ELCH (Explicit Loop Closing Heuristic)

```
pcl::registration::ELCH<PointType> elch;
elch.setReg (icp);
for (int i = 1; i < n; i++)
{
elch.addPointCloud (cloud[i]);
}
elch.setLoopStart (first);
elch.setLoopEnd (last);
elch.compute ();
```



Jochen Sprickerhof:PCL::Registration


pcl_icp -d 0.75 -r 0.75 -i 50 ../data/cloud_0{022..130}.pcd


pcl_elch -d 1.0 -r 1.0 -i 100 ../icp/cloud_0{022..130}.pcd

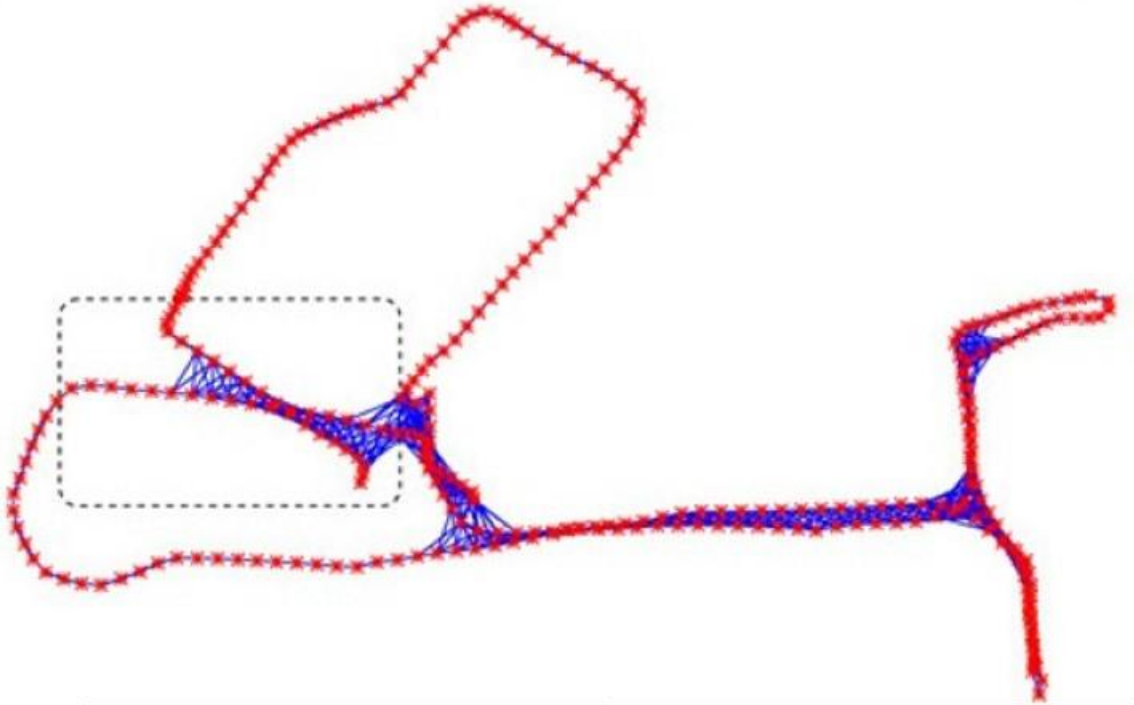
帧间匹配与激光里程计

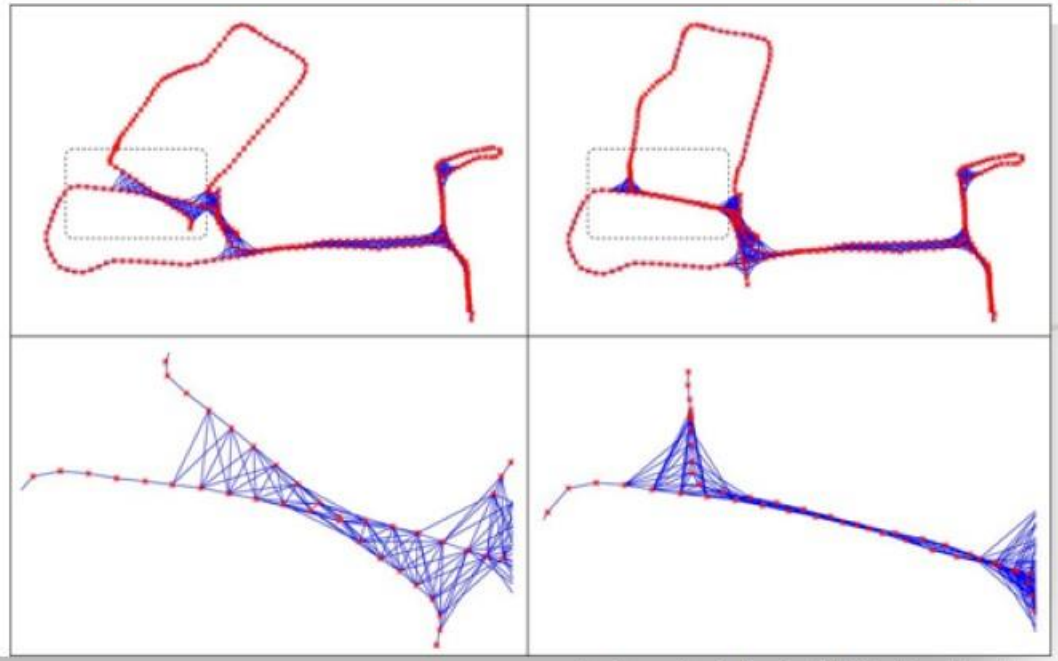
➢ 帧间匹配

PCL SLAM API:

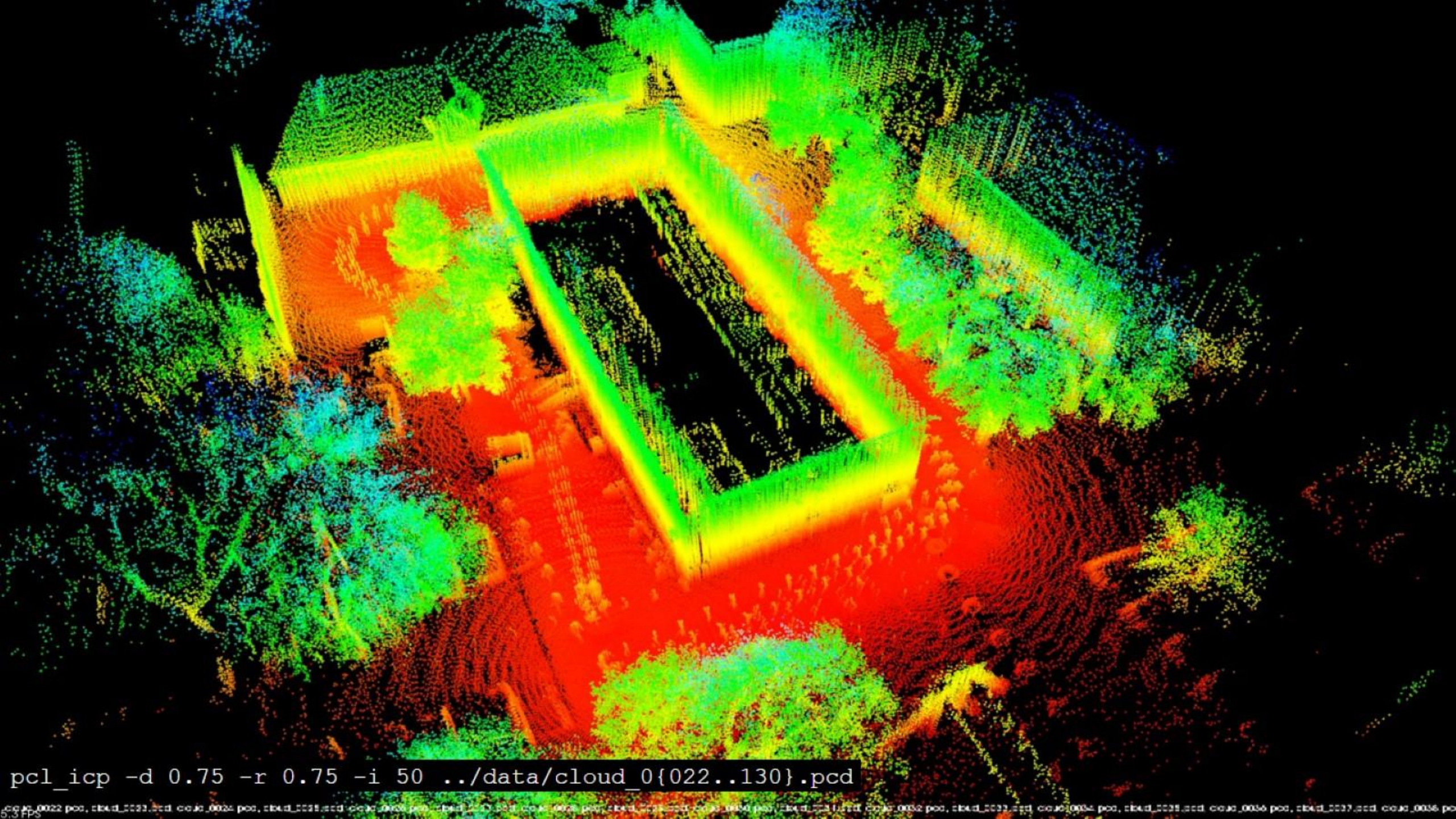
Global Relaxation:LUM

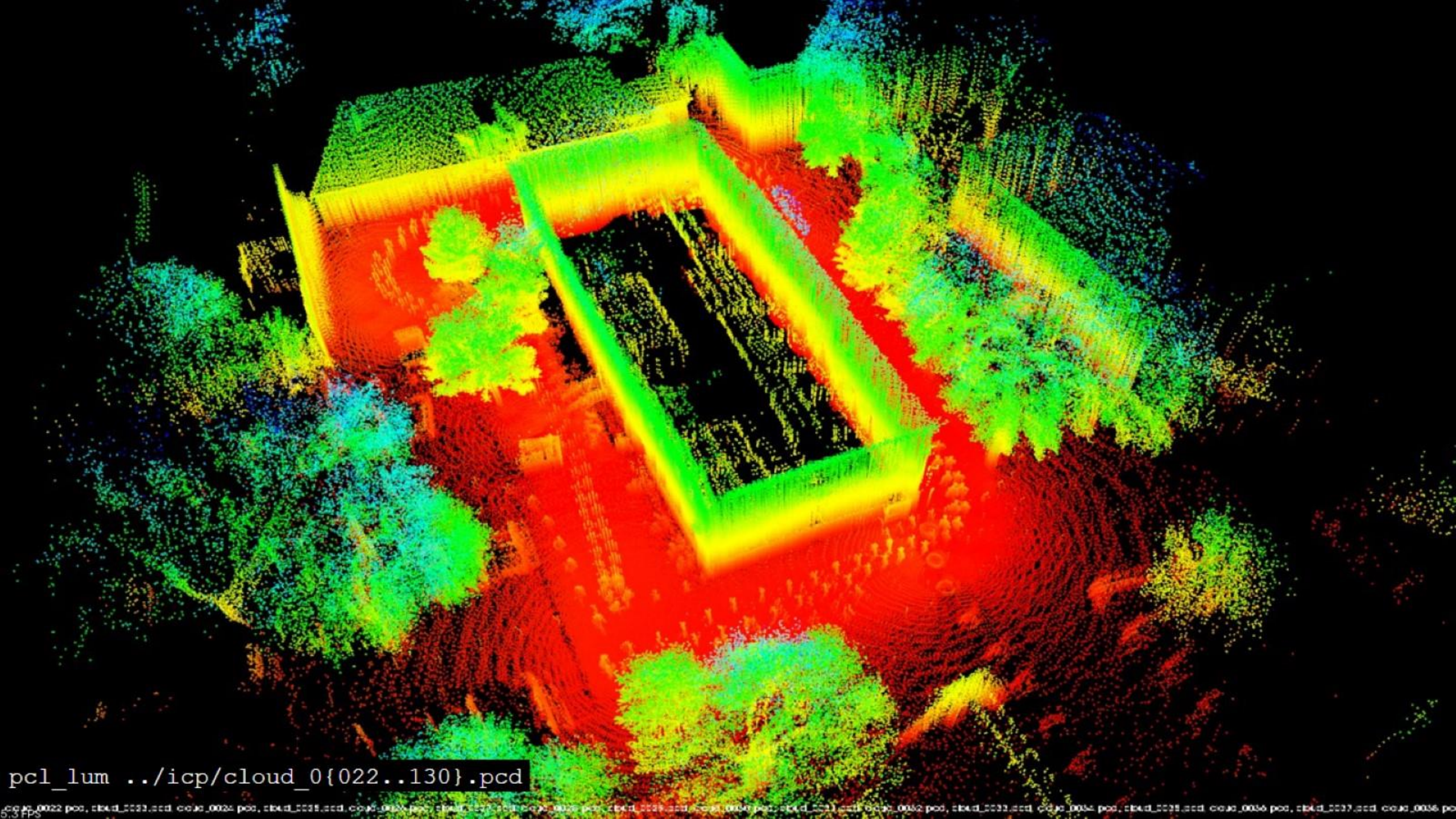
```
pcl::registration::LUM<PointType> lum;
lum.setMaxIterations (lumIter);
lum.setConvergenceThreshold (0.001f);
for (int i = 1; i < n; i++)
{
lum.addPointCloud (cloud[i]);
}
//for all point pairs (i, j)
lum.setCorrespondences (i, j, correspondences);
lum.compute ();
```





Jochen Sprickerhof:PCL::Registration


pcl_icp -d 0.75 -r 0.75 -i 50 ../data/cloud_0{022..130}.pcd

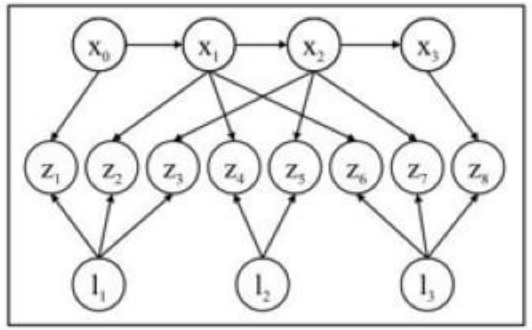

pcl_lum ../icp/cloud_0{022..130}.pcd

内容概要

点云配准方法
SLAM基础框架
帧间匹配与激光里程计
SLAM图优化基础

SLAM图优化基础

- SLAM与贝叶斯网络



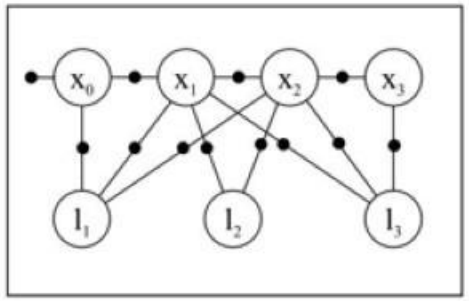
$$P(X, L, Z) = P(x_0) \prod_{i=1}^{M} P(x_i|x_{i-1}, u_i) \prod_{k=1}^{K} P(z_k|x_{i_k}, l_{j_k})$$

$$x_i = f_i(x_{i-1}, u_i) + w_i \quad \Leftrightarrow \qquad z_k = h_k(x_{i_k}, l_{j_k}) + v_k \quad \Leftrightarrow$$

$$P(x_i|x_{i-1}, u_i) \propto \exp -\frac{1}{2} \| f_i(x_{i-1}, u_i) - x_i \|^2_{\Lambda_i} \qquad P(z_k|x_{i_k}, l_{j_k}) \propto \exp -\frac{1}{2} \| h_k(x_{i_k}, l_{j_k}) - z_k \|^2_{\Sigma_k}$$

SLAM图优化基础

- SLAM与因子图



$$\phi_0(x_0) \propto P(x_0)$$

$$P(\Theta) \propto \prod_i \phi_i(\theta_i) \prod_{\{i,j\},i<j} \psi_{ij}(\theta_i, \theta_j)$$

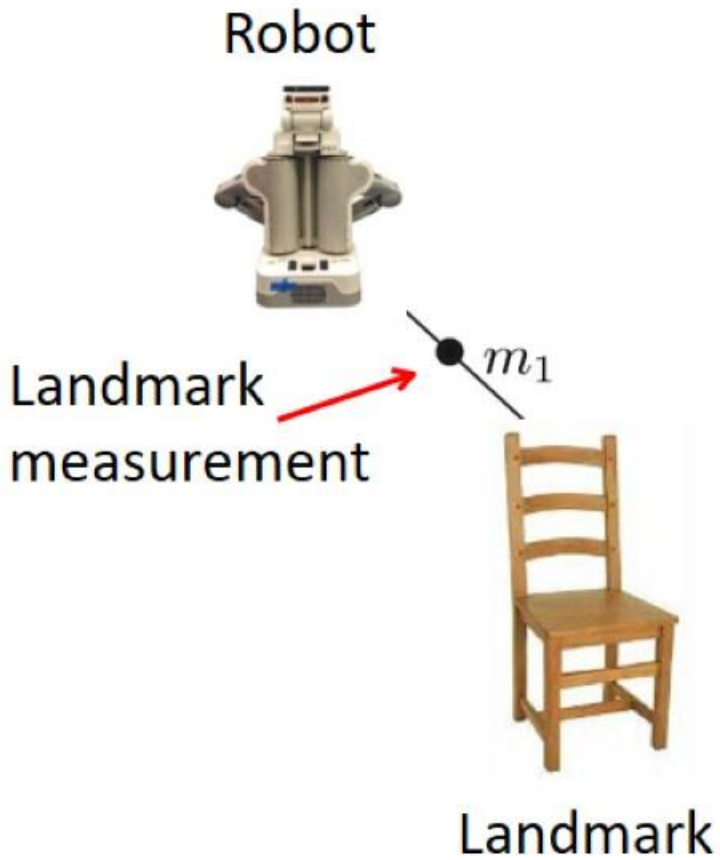
$$\psi_{(i-1)i}(x_{i-1}, x_i) \propto P(x_i | x_{i-1}, u_i)$$

$$\Theta \triangleq (X, L)$$

$$\psi_{i_k j_k}(x_{i_k}, l_{j_k}) \propto P(z_k | x_{i_k}, l_{j_k})$$

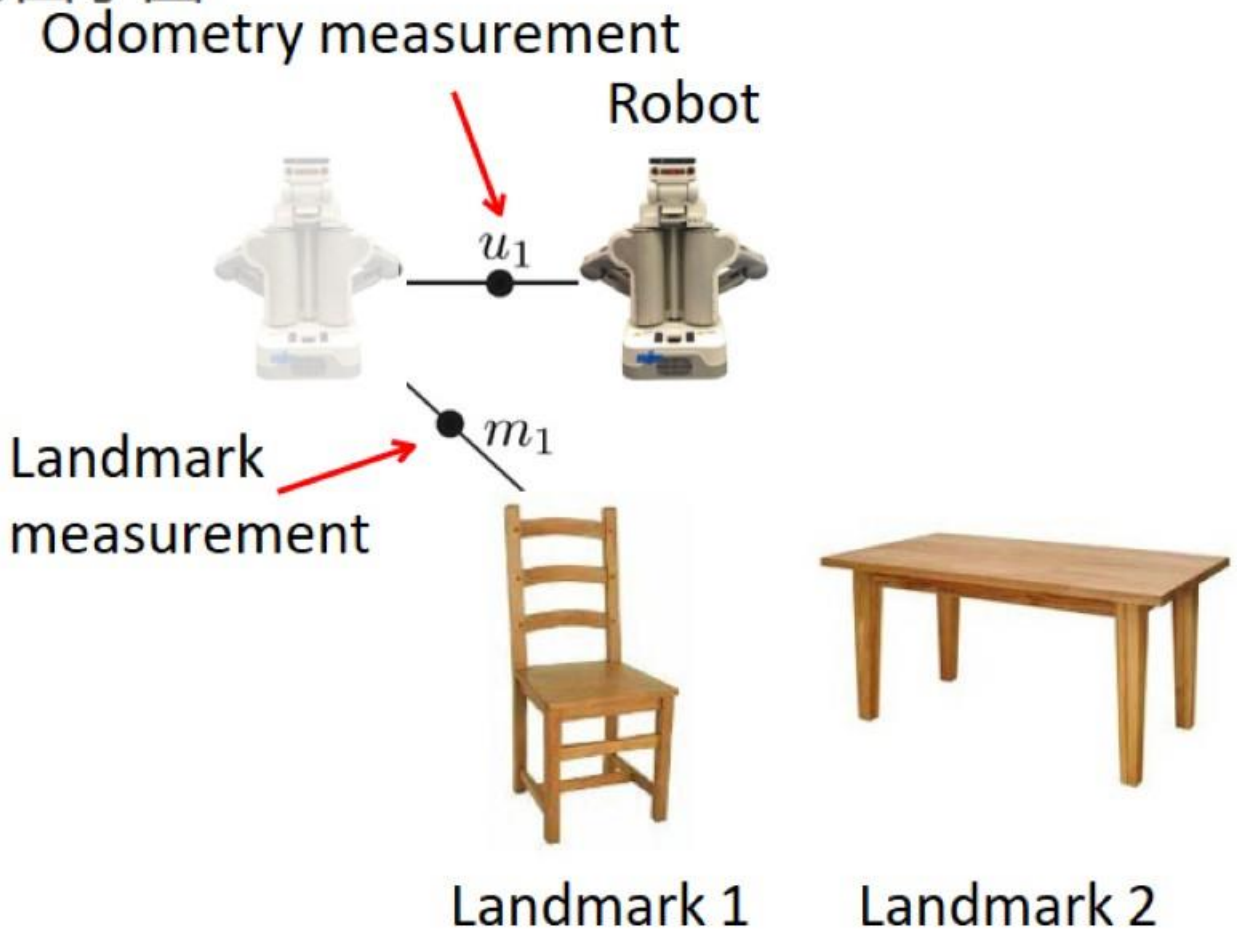
SLAM图优化基础

- SLAM与因子图



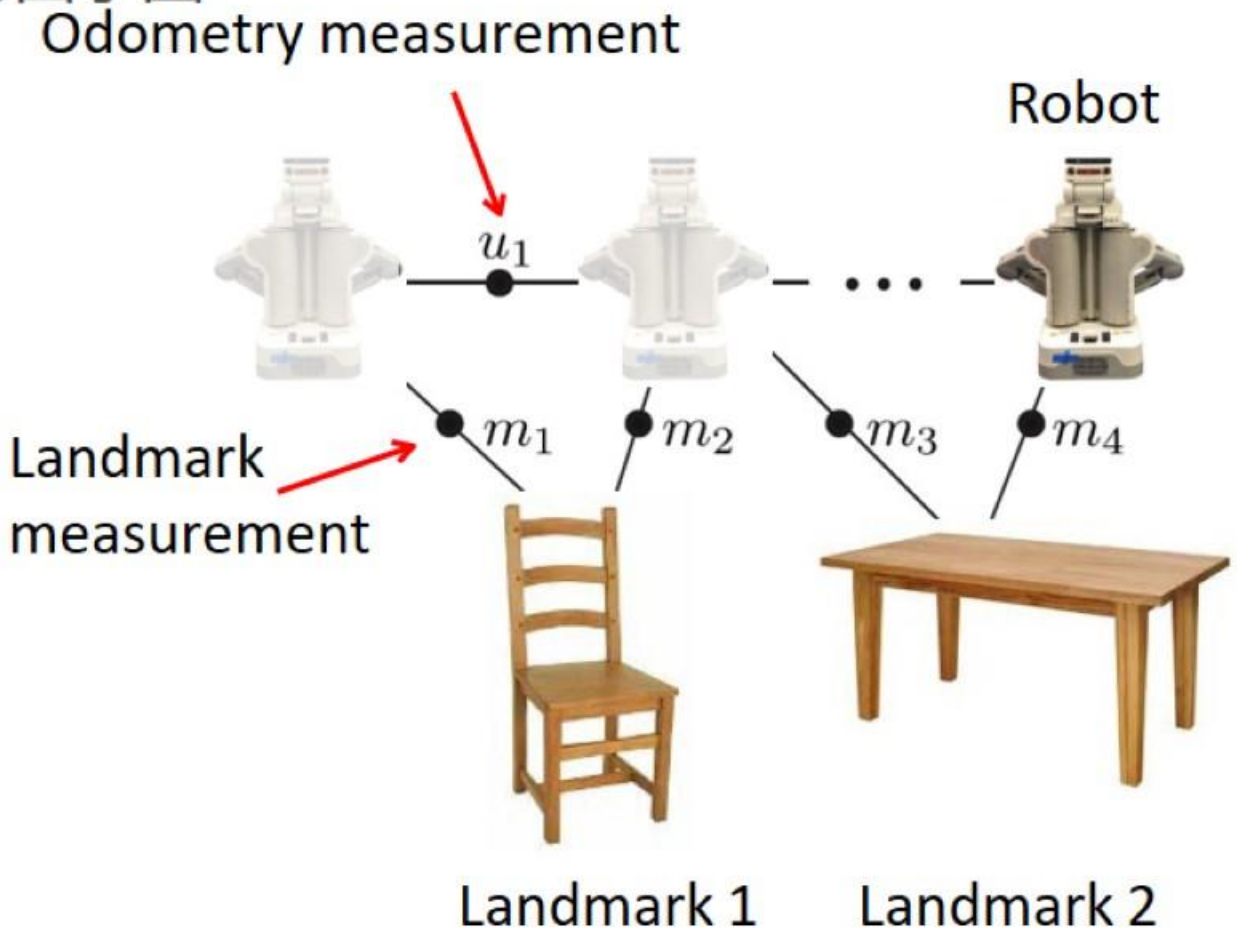
SLAM图优化基础

- SLAM与因子图



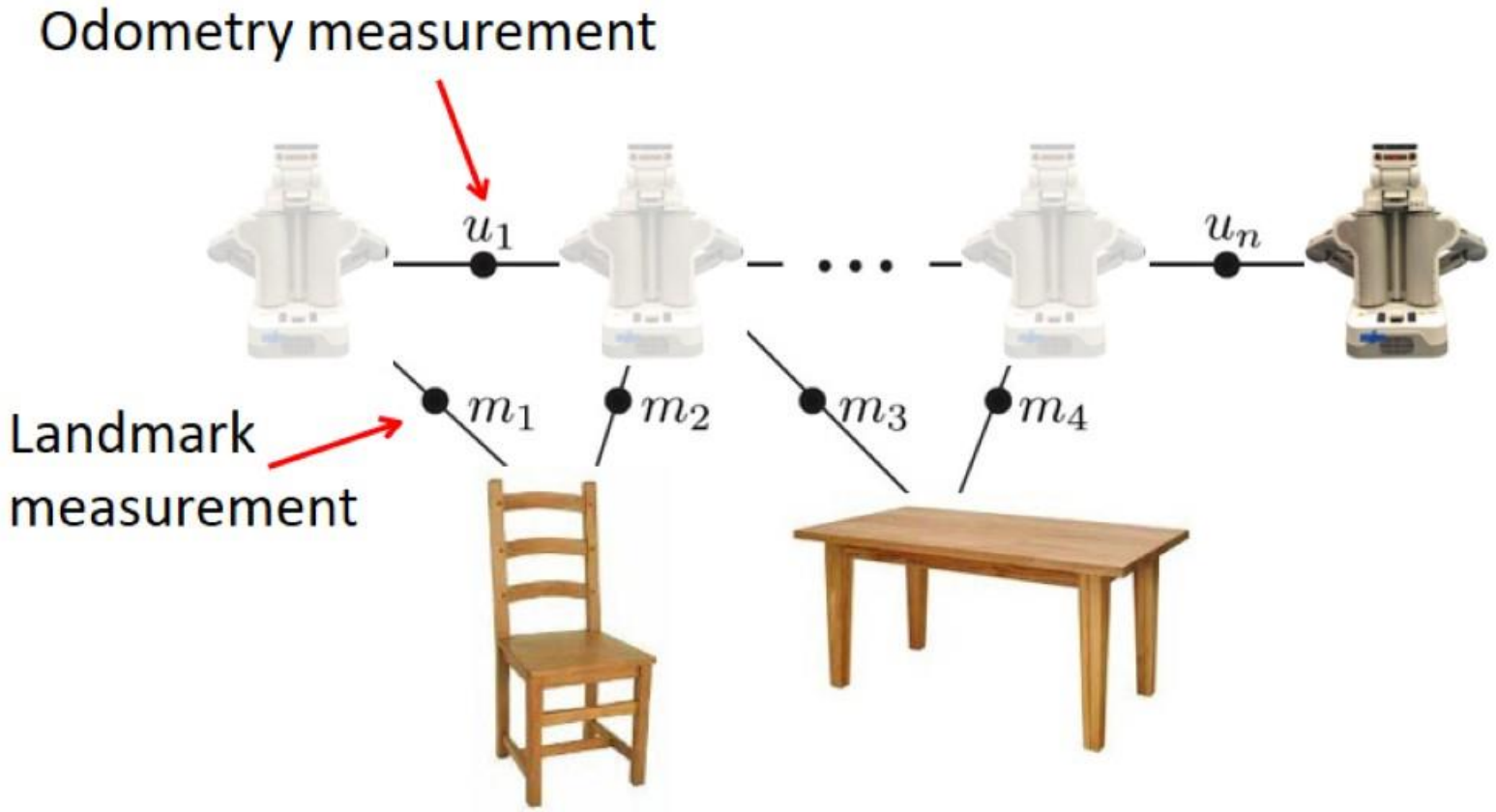
SLAM图优化基础

- SLAM与因子图



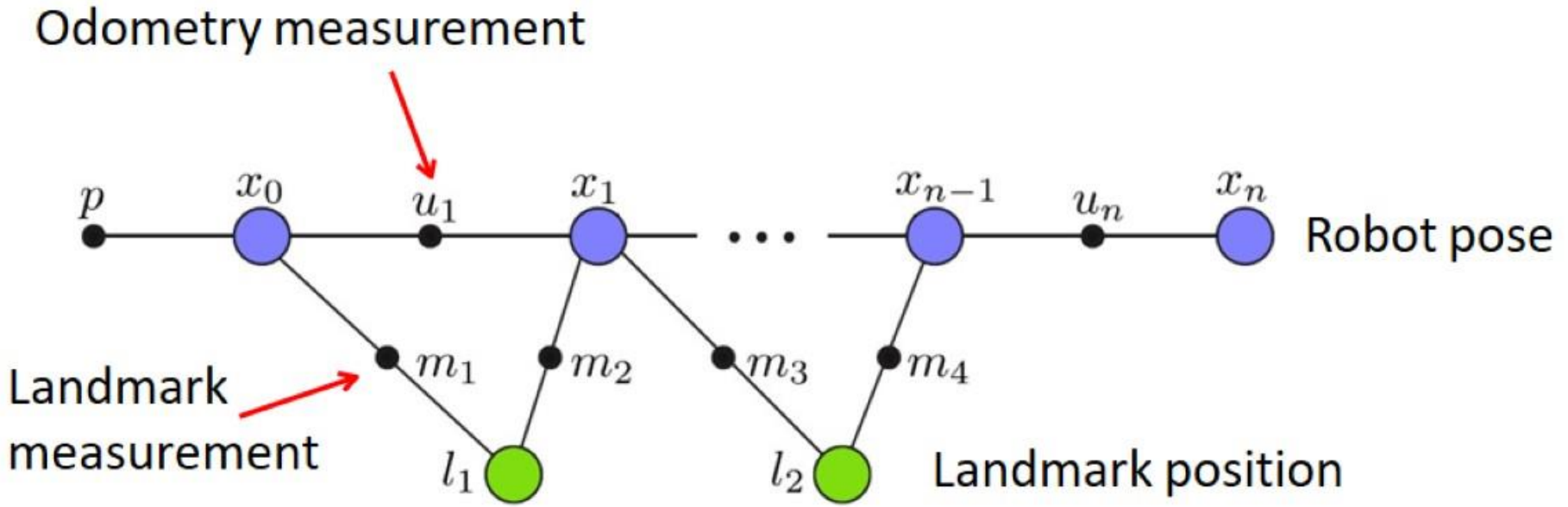
SLAM图优化基础

- SLAM与因子图



SLAM图优化基础

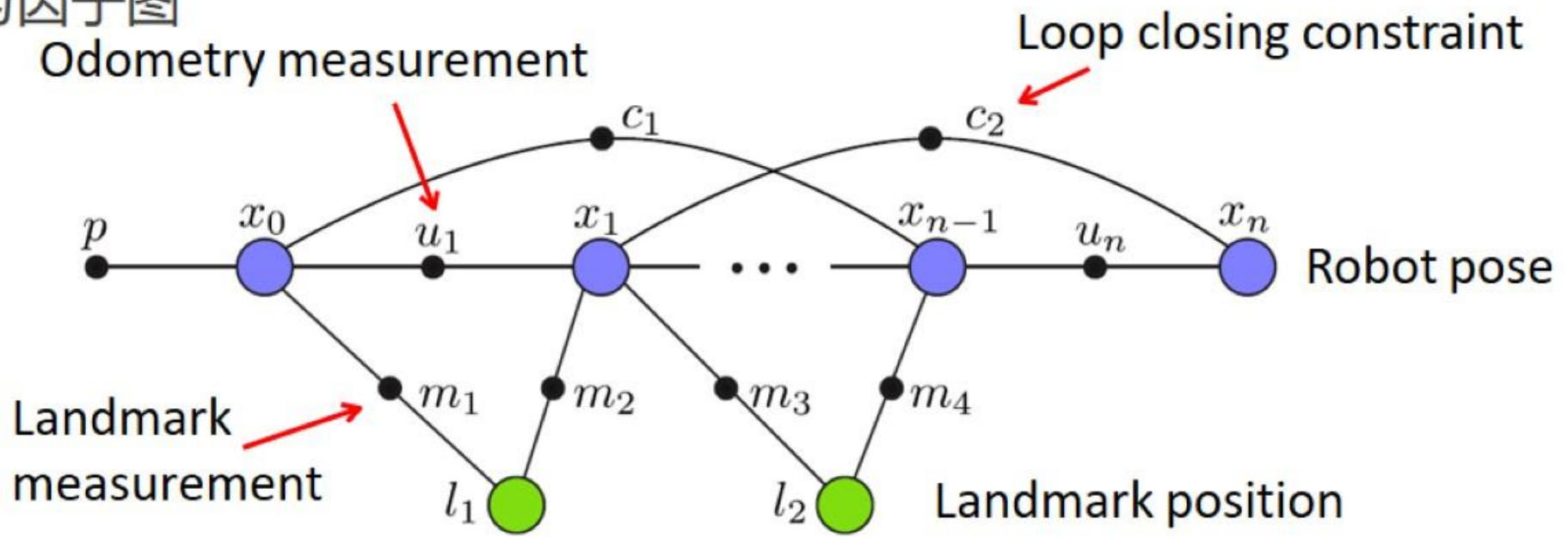
- SLAM与因子图



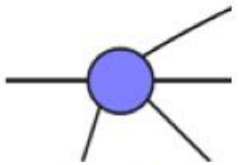
变量节点 (***variable nodes)*** and 因子节点(***factor nodes)***

SLAM图优化基础

- SLAM与因子图



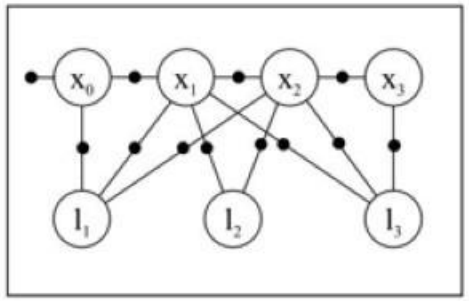
变量节点 (***variable nodes)*** and 因子节点(***factor nodes)***



$$\Theta = \{x_0, x_1 \cdots x_n, l_1, l_2\} \qquad Z = \{p, u_1 \cdots u_n, m_1 \cdots m_4\}$$

SLAM图优化基础

- SLAM与因子图



$$\phi_0(x_0) \propto P(x_0)$$

$$P(\Theta) \propto \prod_i \phi_i(\theta_i) \prod_{\{i,j\},i<j} \psi_{ij}(\theta_i, \theta_j)$$

$$\psi_{(i-1)i}(x_{i-1}, x_i) \propto P(x_i | x_{i-1}, u_i)$$

$$\Theta \triangleq (X, L)$$

$$\psi_{i_k j_k}(x_{i_k}, l_{j_k}) \propto P(z_k | x_{i_k}, l_{j_k})$$

SLAM图优化基础

➢ SLAM与非线性最小二乘

- 最大后验概率估计(MAP)

$$f(\Theta) = \prod_i f_i(\Theta_i)$$

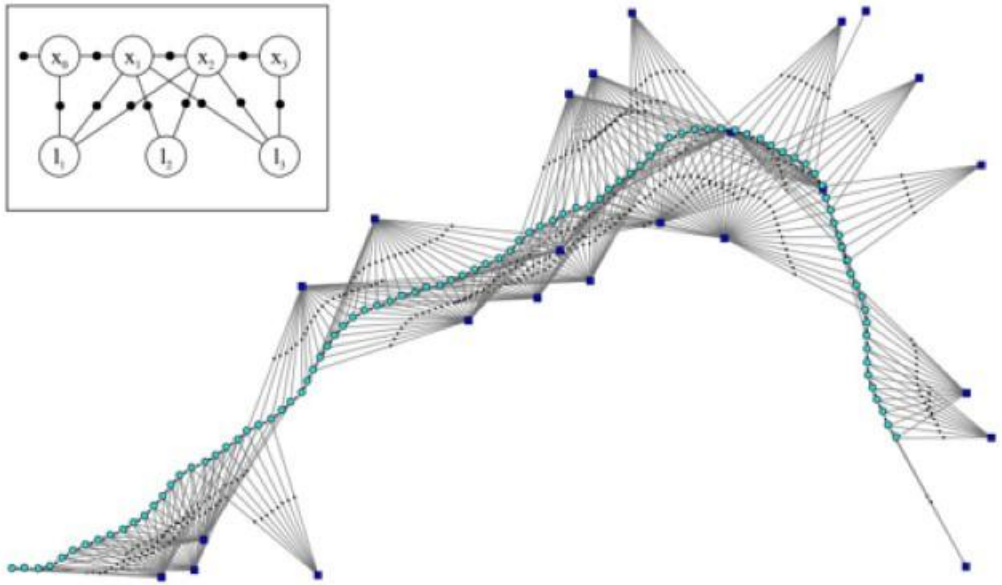
$$\Theta \triangleq (X, L) \quad f_i(\Theta_i) \propto \exp\left(-\frac{1}{2}\|h_i(\Theta_i) - z_i\|^2_{\Sigma_i}\right)$$

$$\Theta^* = \arg\max_{\Theta} f(\Theta)$$

- 负对数转为非线性最小二乘

$$\arg\min_{\Theta}(-\log f(\Theta)) = \arg\min_{\Theta} \frac{1}{2}\sum_i \|h_i(\Theta_i) - z_i\|^2_{\Sigma_i}$$

$$\begin{aligned}
\mathbf{x}^* &= \operatorname*{argmax}_x \prod_i p(\mathbf{z}_i|\mathbf{x}) \\
&= \operatorname*{argmax}_x \prod_i \exp(-\mathbf{e}_i(\mathbf{x})^T \mathbf{\Omega}_i \mathbf{e}_i(\mathbf{x})) \\
&= \operatorname*{argmin}_x \sum_i \mathbf{e}_i(\mathbf{x})^T \mathbf{\Omega}_i \mathbf{e}_i(\mathbf{x})
\end{aligned}$$



$$P(\Theta) \propto \prod_i \phi_i(\theta_i) \prod_{\{i,j\},i<j} \psi_{ij}(\theta_i, \theta_j) \qquad \phi_0(x_0) \propto P(x_0)$$

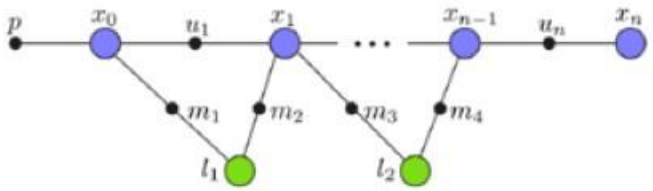
$$\Theta \triangleq (X, L)$$

$$\psi_{(i-1)i}(x_{i-1}, x_i) \propto P(x_i|x_{i-1}, u_i)$$

$$\psi_{i_k j_k}(x_{i_k}, l_{j_k}) \propto P(z_k|x_{i_k}, l_{j_k})$$

SLAM图优化基础

➢ SLAM与因子图



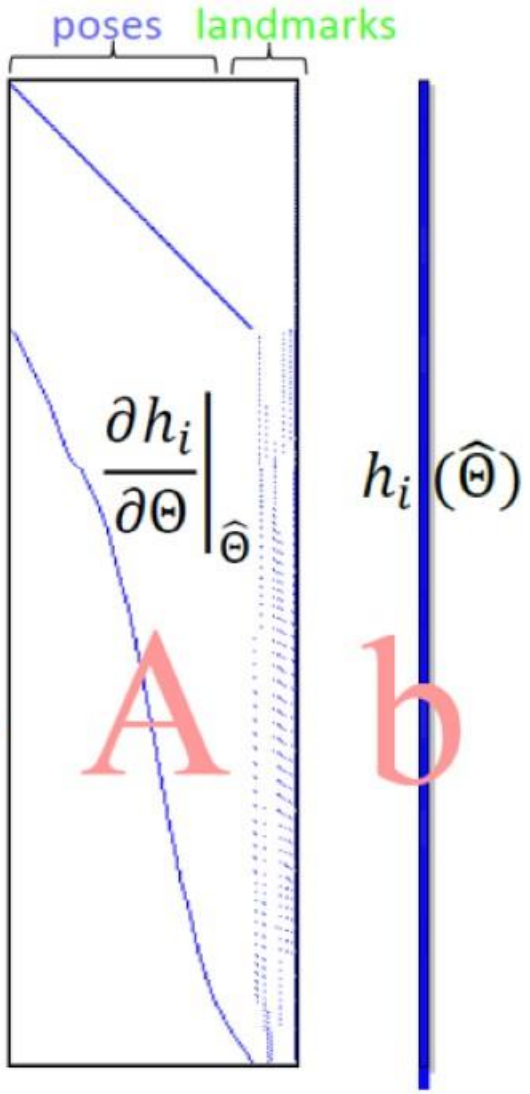
$$\text{argmax}_\Theta \prod_{z \in Z} p\,(z|\Theta)$$

Gaussian noise

$$\text{argmin}_\Theta \sum_i \|h_i(\Theta) - z\|^2$$

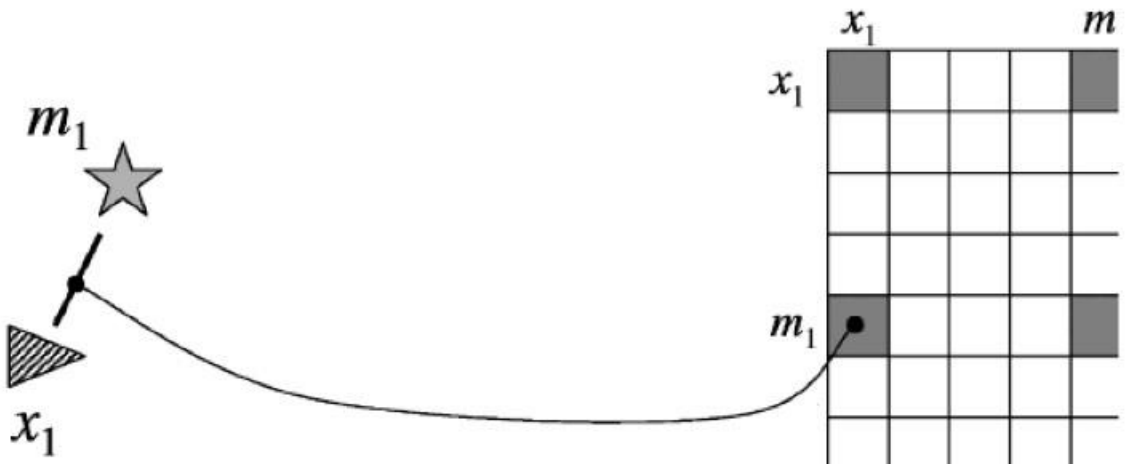
$$\text{argmin}_\theta \|A\theta - b\|^2$$

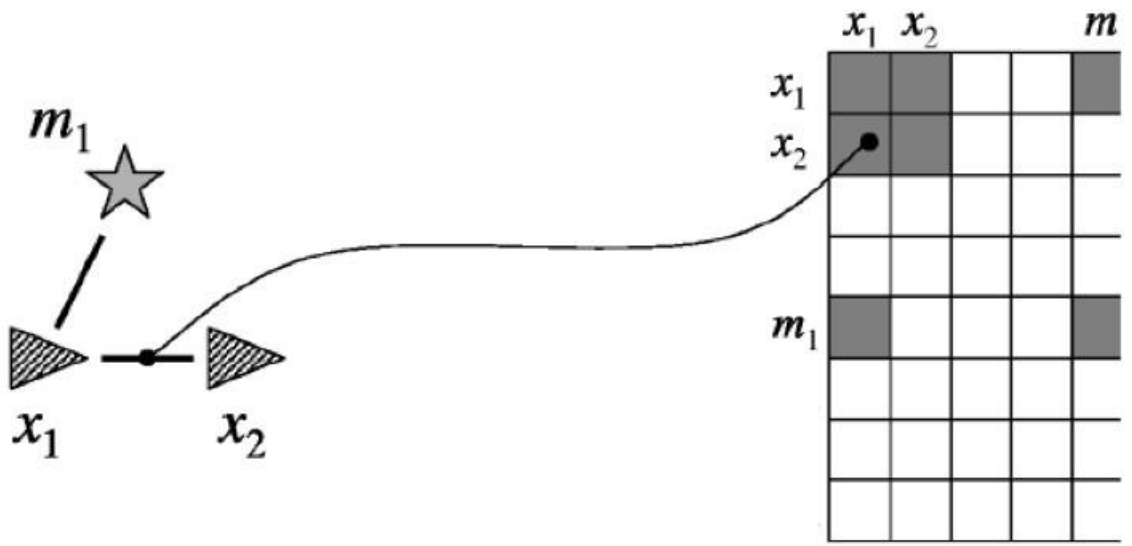
Sparse measurement Jacobian

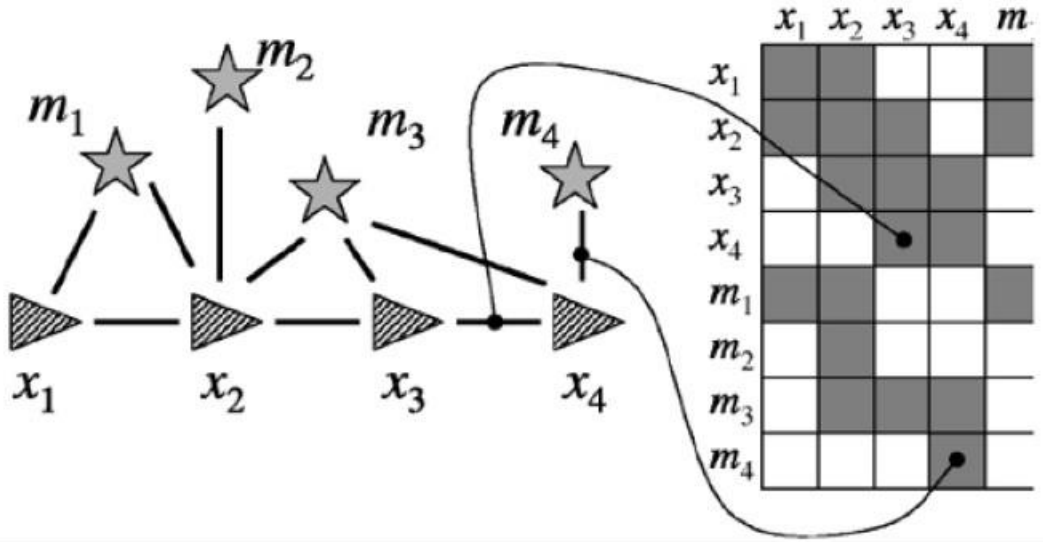


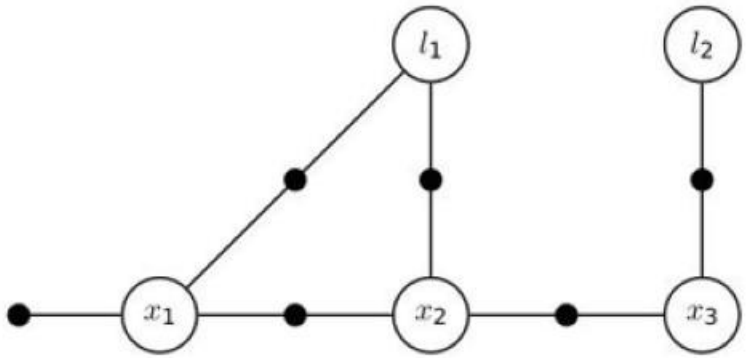
SLAM图优化基础

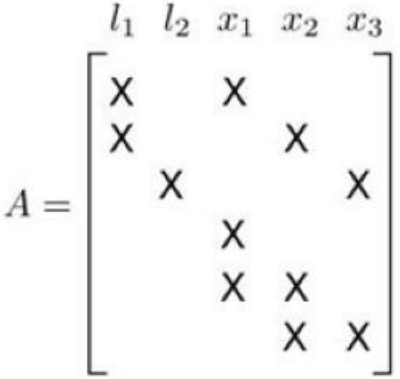
- SLAM与因子图





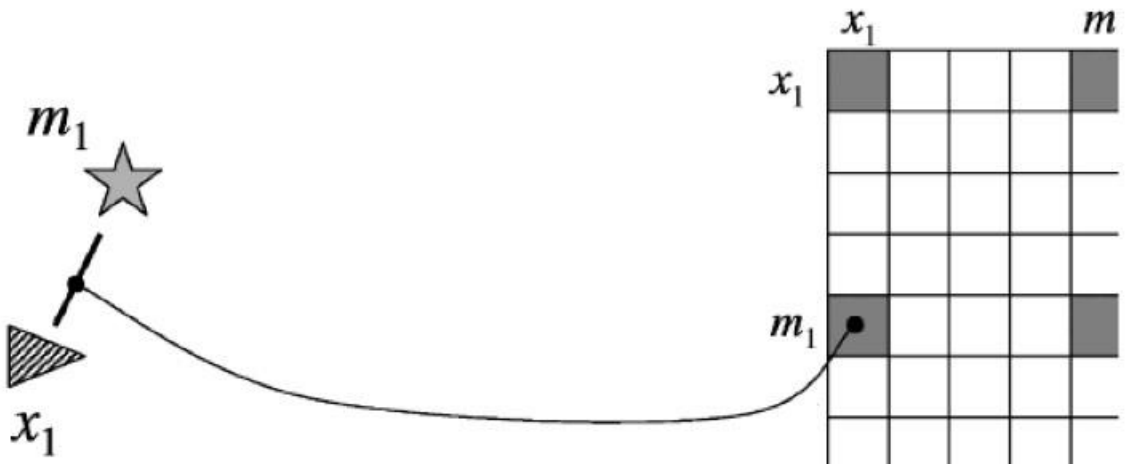


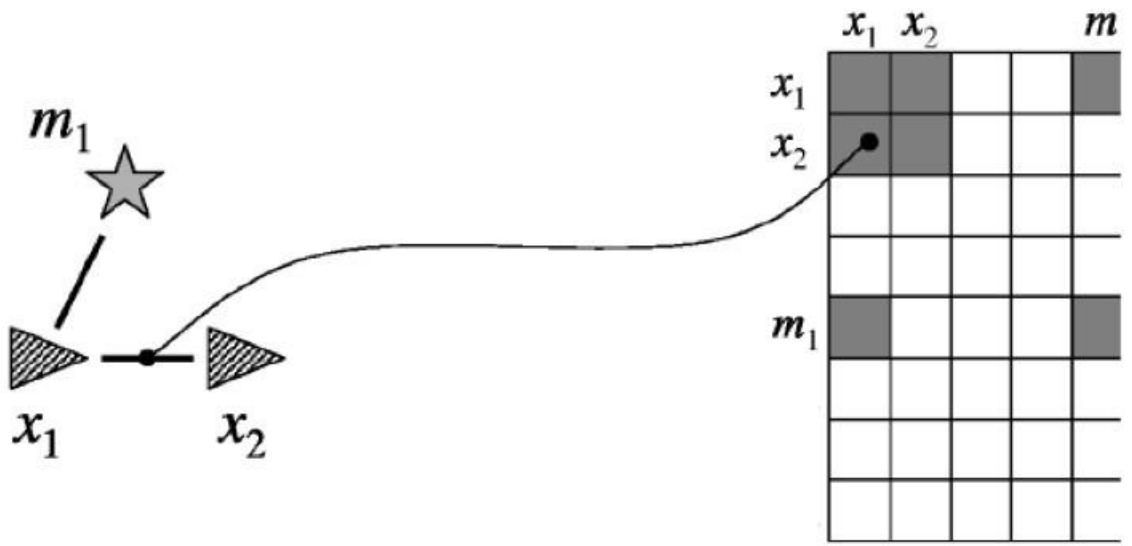


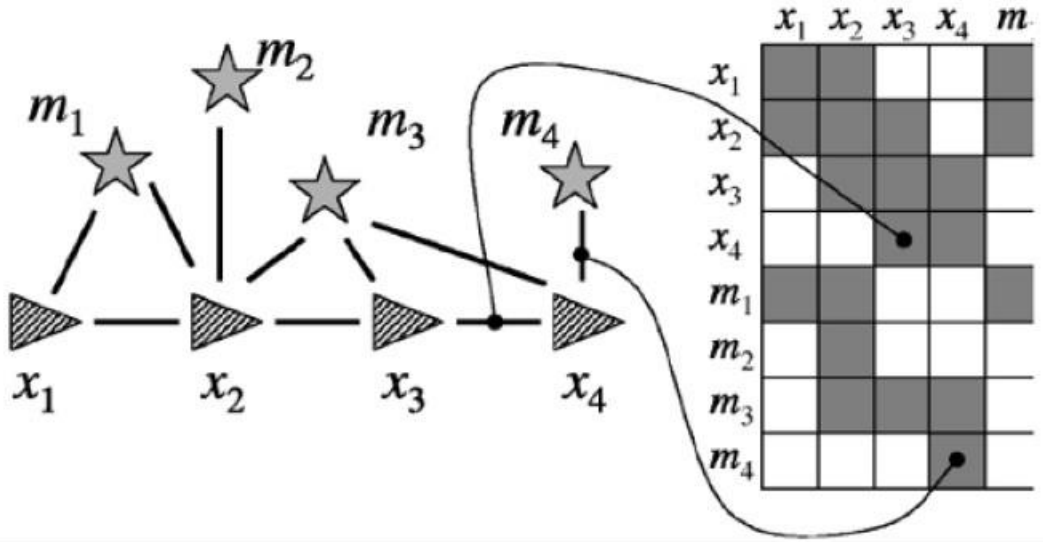


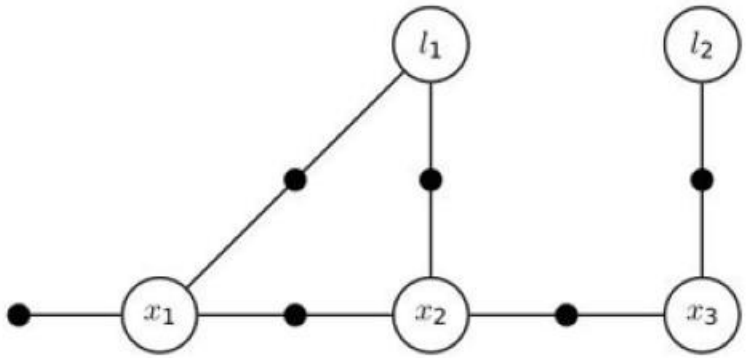
SLAM图优化基础

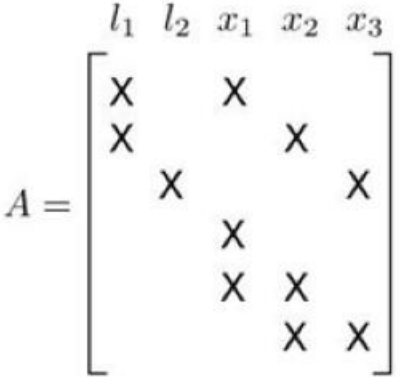
- SLAM与因子图





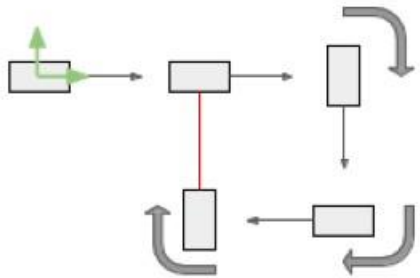




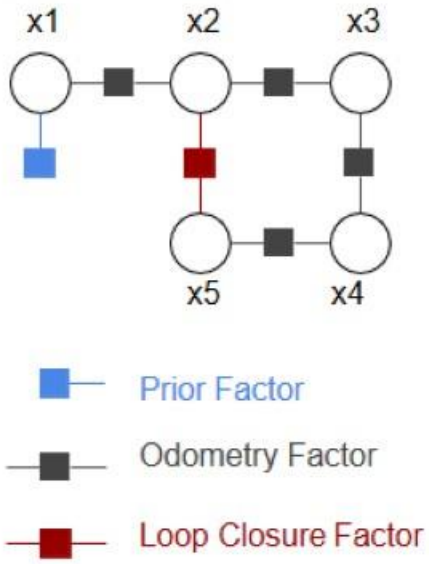


SLAM图优化基础

➢ SLAM图优化-GTSAM

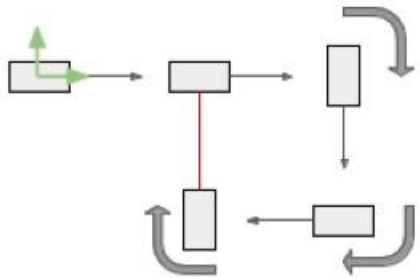


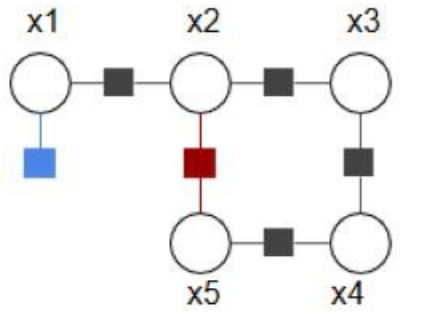
1. 构建因子图(factor graph)
2. 初始化数值 (需要技巧，与实际项目相关，根据实际应用设置)
3. 优化!
4. (可选) 后处理，例如计算边缘分布



SLAM图优化基础

➢ SLAM图优化-GTSAM





Prior Factor

Odometry Factor

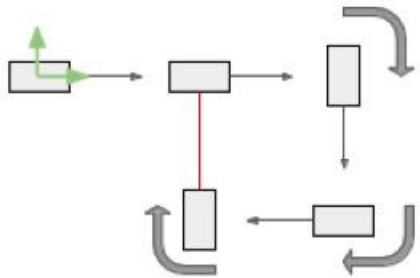
Loop Closure Factor

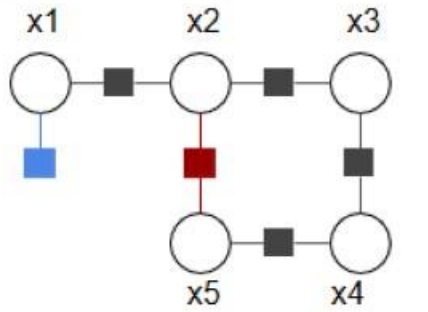
1.构建因子图(factor graph)

```
54  // Create a factor graph container
55  NonlinearFactorGraph graph;
56
57  // Add a prior on the first pose, setting it to the origin
58  // The prior is needed to fix/align the whole trajectory at world frame
59  // A prior factor consists of a mean value and a noise model (covariance matrix)
60  noiseModel::Diagonal::shared_ptr priorModel = noiseModel::Diagonal::Sigmas(Vector3(1.0, 1.0, 0.1));
61  graph.add(PriorFactor<Pose2>(Symbol('x', 1), Pose2(0, 0, 0), priorModel));
62
63  // odometry measurement noise model (covariance matrix)
64  noiseModel::Diagonal::shared_ptr odomModel = noiseModel::Diagonal::Sigmas(Vector3(0.5, 0.5, 0.1));
65
66  // Add odometry factors
67  // Create odometry (Between) factors between consecutive poses
68  // robot makes 90 deg right turns at x3 - x5
69  graph.add(BetweenFactor<Pose2>(Symbol('x', 1), Symbol('x', 2), Pose2(5, 0, 0), odomModel));
70  graph.add(BetweenFactor<Pose2>(Symbol('x', 2), Symbol('x', 3), Pose2(5, 0, -M_PI_2), odomModel));
71  graph.add(BetweenFactor<Pose2>(Symbol('x', 3), Symbol('x', 4), Pose2(5, 0, -M_PI_2), odomModel));
72  graph.add(BetweenFactor<Pose2>(Symbol('x', 4), Symbol('x', 5), Pose2(5, 0, -M_PI_2), odomModel));
73
74  // loop closure measurement noise model
75  noiseModel::Diagonal::shared_ptr loopModel = noiseModel::Diagonal::Sigmas(Vector3(0.5, 0.5, 0.1));
76
77  // Add the loop closure constraint
78  graph.add(BetweenFactor<Pose2>(Symbol('x', 5), Symbol('x', 2), Pose2(5, 0, -M_PI_2), loopModel));
79
80  // print factor graph
81  graph.print("\nFactor Graph:\n");
```

SLAM图优化基础

➢ SLAM图优化-GTSAM





2.初始化数值

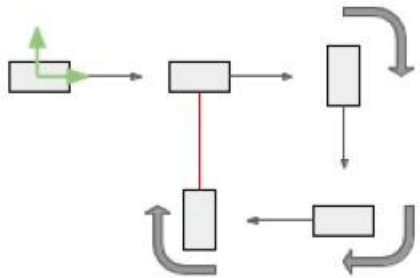
```
84  // initial varible values for the optimization
85  // add random noise from ground truth values
86  Values initials;
87  initials.insert(Symbol('x', 1), Pose2(0.2, -0.3, 0.2));
88  initials.insert(Symbol('x', 2), Pose2(5.1, 0.3, -0.1));
89  initials.insert(Symbol('x', 3), Pose2(9.9, -0.1, -M_PI_2 - 0.2));
90  initials.insert(Symbol('x', 4), Pose2(10.2, -5.0, -M_PI + 0.1));
91  initials.insert(Symbol('x', 5), Pose2(5.1, -5.1, M_PI_2 - 0.1));
92
93  // print initial values
94  initials.print("\nInitial Values:\n");
```

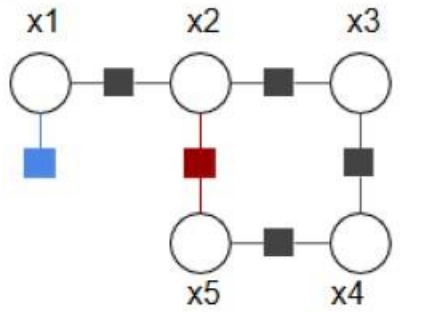
3.优化

```
97  // Use Gauss-Newton method optimizes the initial values
98  GaussNewtonParams parameters;
99
100 // print per iteration
101 parameters.setVerbosity("ERROR");
102
103 // optimize!
104 GaussNewtonOptimizer optimizer(graph, initials, parameters);
105 Values results = optimizer.optimize();
106
107 // print final values
108 results.print("Final Result:\n");
```

SLAM图优化基础

➢ SLAM图优化-GTSAM





Prior Factor

Odometry Factor

Loop Closure Factor

4.后处理边缘分布

```
111   // Calculate marginal covariances for all poses
112   Marginals marginals(graph, results);
113
114   // print marginal covariances
115   cout << "x1 covariance:\n" << marginals.marginalCovariance(Symbol('x', 1)) << endl;
116   cout << "x2 covariance:\n" << marginals.marginalCovariance(Symbol('x', 2)) << endl;
117   cout << "x3 covariance:\n" << marginals.marginalCovariance(Symbol('x', 3)) << endl;
118   cout << "x4 covariance:\n" << marginals.marginalCovariance(Symbol('x', 4)) << endl;
119   cout << "x5 covariance:\n" << marginals.marginalCovariance(Symbol('x', 5)) << endl;
```